TECHPULSE: A REAL-TIME HYBRID RETRIEVAL-AUGMENTED GENERATION SYSTEM FOR EMERGING TECHNOLOGY INTELLIGENCE

Final Report

by

Aye Khin Khin Hpone (Yolanda Lim) st125970

Dechathon Niamsa-Ard st126235

AT82.9002 Selected Topic: Data Engineering and MLOps

Instructor: Dr. Chantri Polprasert

Teaching Assistants: Swaraj Bhanja, Suryansh Srivastava

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ACKNOWLEDGMENTS

We express sincere gratitude to Dr. Chantri Polprasert, the course instructor, for providing guidance on the system design and evaluation methodology throughout this project. We also thank the teaching assistants, Mr. Swaraj Bhanja and Mr. Suryansh Srivastava, for their constructive feedback during development milestones.

ABSTRACT

Large language models produce hallucinated or outdated outputs when queried about recent developments. Retrieval-Augmented Generation (RAG) mitigates this by grounding generation in retrieved evidence, yet the relative benefit of hybrid retrieval over vector-only baselines remains under-evaluated for single-domain corpora.

This study presents TechPulse, a cloud-native Hybrid RAG system for emerging technology intelligence. The system ingests data from five sources (ArXiv, Hacker News, DEV.to, GitHub Trending, RSS feeds) with deterministic preprocessing and MiniLM embedding, indexed in PostgreSQL with pgvector. A five-stage hybrid retrieval pipeline combines metadata filtering, vector similarity search, full-corpus BM25 keyword search, weighted Reciprocal Rank Fusion (RRF; vector 0.50, BM25 0.35, recency 0.15), and cross-encoder reranking (ms-marco-MiniLM-L-6-v2). An automatic LLM fallback chain, dual-criteria drift detection, and three-layer hallucination verification ensure availability and trustworthiness. A 100-sample paired evaluation (50 baseline, 50 hybrid) with RAGAS metrics, Wilcoxon signed-rank testing, and Cohen's d effect size shows that hybrid retrieval outperforms baseline with a composite quality gain of $+0.047$ (0.723 vs. 0.676), driven by faithfulness $+0.058$, answer relevancy $+0.065$, and context precision $+0.070$, at the cost of a small latency increase (mean $1.13\,\mathrm{s} \rightarrow 2.49\,\mathrm{s}$; Cohen's $d = -0.34$, small effect). Drift detection achieves 100% accuracy under four simulated drift scenarios. Monthly cost is \$0.33 at 100 queries/day, well within the AWS Free Tier. The results demonstrate that weighted RRF with cross-encoder reranking measurably improves retrieval quality for a single-domain corpus while preserving architectural extensibility for heterogeneous workloads.

Keywords: Retrieval-Augmented Generation; Hybrid Information Retrieval; Vector Databases; Cloud-Native Architecture; MLOps; Semantic Embedding; Emerging Technology Intelligence.

CONTENTS

ACKNOWLEDGMENTS **i**

ABSTRACT **ii**

1 Introduction **1**

1.1 Problem Description and Motivation 1
1.2 Research Questions 2
1.3 Objectives of the Study 2
1.4 Significance of the Study 2
1.5 Alignment Between Research Questions and Evaluation 3

2 Background and Related Work **4**

2.1 Retrieval-Augmented Generation 4
2.2 Embedding Models and Vector Search 4
2.3 Hybrid Retrieval and Reranking 4
2.4 RAG Evaluation Methodologies 5
2.5 Data and Concept Drift in Retrieval Systems 5
2.6 Hallucination Detection and Faithfulness Verification 5
2.7 MLOps and Production Machine Learning Systems 6
2.8 Research Gap 6

3 Methodology **8**

3.1 Overview of Technical Approach 8
3.2 Why This Approach is Suitable 8
3.3 Technical Contributions and Design Decisions 9
3.4 Baseline Retrieval Configuration 10
3.5 Implemented Hybrid Retrieval Configuration 10
3.6 Hallucination Verification Strategy 12
3.7 Optimization Objective 12
3.8 Tool Selection and Implementation Stack 13
3.9 System Context Overview 14

4 System Architecture and Pipeline Design **16**

4.1 Architectural Overview 16
4.2 Alignment with the AWS Well-Architected Framework 17
4.3 Data Sources 17
4.4 Data Engineering Pipeline 18
4.5 Embedding and Indexing 20
4.6 Storage Architecture (Medallion Layout) 21
4.7 Database Schema 21

4.8 Prompt Construction and Grounded Generation 22
4.9 LLM Backend Fallback Chain 23
4.10 Deployment, Monitoring, and MLOps 23
4.11 AWS Cloud Deployment Architecture 26
4.12 User Interface 27

5 Experimental Plan and Evaluation Metrics 28
5.1 Evaluation Objective 28
5.2 Experimental Configurations 28
5.3 Evaluation Dataset 29
5.4 Evaluation Metrics 30
5.5 Analysis and Discussion Framework 35
5.6 Result Visualizations 36

6 Implementation Details 43
6.1 Overall Implementation Status 43
6.2 Key Enhancements Beyond the Original Proposal 43
6.3 AWS Free Tier Deployment 43
6.4 Test Coverage 46
6.5 Experiment Tracking and Result Versioning 46
6.6 Custom CloudWatch Metrics 46
6.7 Engineering Observations 47
6.8 System Demonstration 47

7 Conclusions and Future Work 51
7.1 Summary of Contributions 51
7.2 Project Phases 51
7.3 Known Limitations 52
7.4 Project Timeline 53
7.5 Future Extensions 54

REFERENCES 56

A Source Code Availability 59

B System Setup and Reproduction Steps 60

LIST OF FIGURES

3.1 Retrieval strategies compared during design: baseline vector-only (left), weighted hybrid with $\alpha/\beta/\gamma$ reranking and p95 = 25 s tail latency (centre, not selected), and weighted BM25+RRF hybrid pipeline with cross-encoder reranking selected for the production system (right). 12
3.2 System context diagram for TechPulse. Solid arrows represent data and query flows; dashed arrows represent monitoring and evaluation signals. 15

4.1 AWS architecture diagram: service-level deployment topology for TechPulse (Free Tier). 16
4.2 Six-layer conceptual architecture of TechPulse. Right-hand arrows show the upward data flow; the left-hand dashed arrow shows the query path. Layer 4 uses weighted BM25+RRF fusion with cross-encoder reranking, selected over weighted $\alpha/\beta/\gamma$ linear reranking based on design-phase analysis (Section 5.4.5). Layer 5 (Generative Synthesis) implements the LLM backend selection and fallback chain described in Section 4.9. 16
4.3 End-to-end data engineering and serving pipeline. Solid arrows indicate data flow; dashed arrows indicate monitoring signals. 18
4.4 Document state machine with four deterministic states. Transition labels show triggering processes (top) and storage destinations (bottom). 19
4.5 Entity–relationship diagram of the TechPulse PostgreSQL schema. 22
4.6 MLOps lifecycle loop with continuous ingestion, evaluation, monitoring, and budget governance. 24
4.7 AWS cloud deployment architecture (AWS Free Tier configuration). 26

5.1 Evaluation framework: four measurement dimensions with target thresholds, mapped to research questions. 30
5.2 Baseline vs. hybrid retrieval comparison across four evaluation dimensions. . . 37
5.3 Design-phase grid search: reranking weight (α) vs. mean similarity and p95 latency. The grid search used a reduced query subset; when the best configuration (α=0.70) was evaluated on the full 50-query set, p95 latency rose to 25 s (see Section 5.5), confirming that linear scoring did not scale and motivating the switch to weighted RRF. 38
5.4 Per-source retrieval distribution for baseline vs. hybrid retrieval modes. 39

5.5 Mean end-to-end latency by query category for baseline and hybrid retrieval. . . 39
5.6 Multi-metric radar comparison of baseline and hybrid retrieval. 40
5.7 Paired-difference histogram of per-query latency shift (hybrid − baseline). . . . 40

6.1 TechPulse frontend: overview page with query input and retrieval mode selection. 48
6.2 TechPulse frontend: hybrid vs. baseline retrieval for the same query. 48
6.3 GitHub Actions CI/CD pipeline: workflow run executing successfully. 49
6.4 AWS CloudFormation console: TechPulse stack with all resources successfully created. 49
6.5 AWS RDS console: TechPulse PostgreSQL instance with pgvector extension. . 50

7.1 Project timeline (Gantt chart). All four phases completed. 54

LIST OF TABLES

1.1 Mapping of Research Questions to Evaluation Metrics and Methodology. 3

2.1 Comparison of Related Work and Positioning of TechPulse. 7

3.1 Implemented Tools, Services, and Justification. 14

4.1 TechPulse Alignment with the AWS Well-Architected Framework Six Pillars. . 17
4.2 Implemented Data Sources and Roles. 18
4.3 Document State Transition Model. 19
4.4 Preprocessing Pipeline: 17-Step Text Normalization. 20
4.5 Storage Layers and Responsibilities (Medallion Architecture). 21
4.6 Implemented LLM Backend Fallback Hierarchy. 23
4.7 Implemented LLM Generation Parameter Configuration. 24

5.1 Experimental Configuration Comparison. 29
5.2 Summary of Evaluation Metrics and Targets. 34
5.3 Evaluation Summary Dashboard: Baseline vs. Hybrid with Target Thresholds. . 41

6.1 Component Implementation Status Summary. 44
6.2 Estimated Monthly Cost Breakdown at 100 Queries/Day Workload. 45

7.1 Project Phase Summary. 52

CHAPTER 1

INTRODUCTION

1.1 Problem Description and Motivation

The exponential growth of emerging technologies—artificial intelligence, machine learning, distributed systems, and advanced computing—has produced technical knowledge fragmented across heterogeneous platforms: scholarly repositories (ArXiv), community forums (Hacker News), developer blogs (DEV.to), and open-source ecosystems (GitHub). This fragmentation makes timely synthesis increasingly difficult for technology professionals.

Purely generative large language model (LLM) assistants are insufficient for emerging technology intelligence tasks due to their static knowledge cutoff, susceptibility to hallucinated responses, lack of verifiable evidence grounding, and inability to integrate with continuously updating data streams (Ji et al., 2023). Although Retrieval-Augmented Generation (RAG) architectures partially address these limitations by incorporating external context, the relative benefit of augmenting vector-only retrieval with lexical and recency signals remains under-evaluated in cost-constrained, single-domain settings. Moreover, baseline vector-only systems typically lack temporal relevance signals, structured metadata constraints, and recency-aware ranking mechanisms, which are important for accurately capturing rapidly evolving technology trends.

A further gap exists at the systems level: many RAG research prototypes focus primarily on retrieval accuracy without addressing production-readiness—deterministic pipeline design, idempotent ingestion, monitoring and observability, deployment automation, and cloud cost governance are frequently neglected, limiting their applicability in real-world operational environments.

The central engineering and research challenge addressed in this project is therefore:

> ***How can a scalable, continuously updating hybrid retrieval system be designed and deployed to synthesize emerging technology trends from heterogeneous scholarly and industry sources while satisfying latency and cost constraints?***

This project is motivated by three converging factors. First, technology professionals increasingly require timely and citation-grounded intelligence that synthesizes insights from both scholarly research and rapidly evolving industry discussions. Second, mature serverless cloud infrastructures now enable cost-effective deployment of production-grade data and machine learning pipelines. Third, a gap remains in the Retrieval-Augmented Generation (RAG) literature between retrieval-focused research prototypes and operationally robust systems capable of

continuous ingestion, evaluation, and deployment in real-world environments.

1.2 Research Questions

To address the central problem, this study investigates the following research questions:

1. **Data Engineering.** How can a cloud-native pipeline reliably ingest heterogeneous scholarly and industry data in a deterministic, idempotent manner?
2. **Retrieval Optimization.** How does hybrid retrieval (metadata filtering, BM25, and Reciprocal Rank Fusion) compare to baseline vector-only retrieval in context precision and faithfulness?
3. **Operational Trade-offs.** What trade-offs arise between retrieval precision, p95 latency, and cost per query?
4. **MLOps Integration.** How can deployment automation, monitoring, and cost governance ensure the reliability of a production-grade RAG system?

1.3 Objectives of the Study

The primary objective is to design, implement, and evaluate a hybrid Retrieval-Augmented Generation system capable of synthesizing real-time emerging technology intelligence within a scalable, cost-aware cloud-native architecture. In this project, machine learning model development is approached from a systems perspective: rather than training a large model from scratch, the work focuses on selecting appropriate pretrained models, integrating them into a retrieval-augmented architecture, tuning retrieval and ranking behavior through systematic ablation, and evaluating quality, latency, and cost trade-offs in a production-like deployment setting. The specific objectives are:

1. To construct a cloud-native data engineering pipeline for batch and near-real-time ingestion from five heterogeneous data sources.
2. To implement and compare baseline vector-only retrieval against hybrid retrieval with metadata filtering, full-corpus BM25, and Reciprocal Rank Fusion (RRF).
3. To evaluate system performance across context precision, faithfulness, p95 retrieval latency, and cost per query.
4. To deploy and monitor the system using serverless AWS infrastructure consistent with MLOps best practices (Sculley et al., 2015; Breck et al., 2017).

1.4 Significance of the Study

This project contributes at the intersection of data engineering, natural language processing, and MLOps in three ways. First, it provides controlled empirical evidence that hybrid retrieval

(weighted BM25+RRF with cross-encoder reranking) measurably improves over vector-only baselines within a single-domain, cost-constrained setting—a configuration common in practice but under-represented in the literature. Second, it demonstrates that weighted Reciprocal Rank Fusion combined with a lightweight cross-encoder (ms-marco-MiniLM-L-6-v2) achieves a composite quality gain of $+0.047$ over the baseline while maintaining a manageable latency trade-off (mean $+1.36$ s, Cohen's $d = -0.34$, small effect). Third, the end-to-end system integrates hybrid retrieval, drift detection, hallucination verification, and cost governance within a single deployable architecture on the AWS Free Tier, providing a practical blueprint for student and small-team RAG deployments.

1.5 Alignment Between Research Questions and Evaluation

Table 1.1 provides an explicit mapping from each research question to the corresponding evaluation metric and experimental configuration used to answer it.

Table 1.1. Mapping of Research Questions to Evaluation Metrics and Methodology.

RQ	Question (abbreviated)	Primary Metric	Evaluation Approach
1	Cloud-native deterministic ingestion	Pipeline idempotency; SHA-256 deduplication pass rate	Unit tests (24 ingestion tests); repeated pipeline runs
2	Hybrid vs. baseline retrieval precision	Context precision (RAGAS); faithfulness; citation grounding > 0.80	Paired comparison over 50 queries; Wilcoxon signed-rank test; Cohen's d; bootstrap 95% CI
3	Latency and cost trade-offs	p95 latency $<$ 2s (retrieval stage); cost $<$ USD 15/month	Per-query timing; token usage profiling; cost projection
4	MLOps integration	Drift detection coverage; CI/CD pass rate; health-check uptime	Drift simulation runs; CloudWatch alarm verification

The study is structured as follows: Chapter 2 reviews the relevant literature; Chapter 3 presents the methodology and hypotheses; Chapter 4 describes the system architecture; Chapter 5 defines the evaluation framework and presents results; Chapter 6 documents implementation details; and Chapter 7 presents conclusions, limitations, and future work. The following chapter reviews the technical foundations underpinning these design decisions.

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. (2020) as a framework for augmenting sequence-to-sequence generation with a dense retrieval component, enabling the model to access external documents at inference time. This approach significantly improves performance on knowledge-intensive tasks by conditioning generation on retrieved evidence rather than relying solely on parametric memory. Despite their scale, LLMs remain prone to hallucination when responding to domain-specific or time-sensitive queries (Ji et al., 2023), making retrieval-based augmentation a practically important technique.

Recent surveys (Gao et al., 2023; Zhang et al., 2024) categorize RAG systems into four architectural types: (1) single-stage dense retrieval; (2) hybrid lexical–semantic retrieval; (3) multi-stage retrieval with reranking; and (4) iterative or self-reflective retrieval. While baseline dense retrieval improves factual grounding, empirical benchmarks such as BEIR (Thakur et al., 2021) demonstrate that semantic similarity alone is insufficient for high-precision retrieval in dynamic or heterogeneous corpora. Foundational retrieval-augmented systems such as Atlas (Izacard et al., 2022) and RETRO (Borgeaud et al., 2022) demonstrate retrieval augmentation at scale but require computationally intensive pretraining pipelines unsuited to cost-constrained production environments.

2.2 Embedding Models and Vector Search

Dense retrieval relies on embedding models that transform text into high-dimensional vector representations. Sentence-transformer architectures such as MiniLM (Wang et al., 2020) provide computationally efficient embeddings with favorable trade-offs between semantic expressiveness and inference latency. PostgreSQL with the pgvector extension enables cosine similarity search alongside SQL-based metadata filtering within a single relational storage layer, reducing architectural complexity compared to dedicated vector databases such as FAISS or Milvus. This integrated approach is increasingly favored for production RAG deployments (Gao et al., 2023).

2.3 Hybrid Retrieval and Reranking

Hybrid retrieval combines lexical matching (e.g., BM25 (Best Matching 25) or keyword filtering) with semantic similarity search to improve retrieval precision. Thakur et al. (2021) demonstrate that hybrid approaches consistently outperform purely dense retrieval across heterogeneous benchmarks. Reranking further refines candidate relevance using additional contextual signals.

Cross-encoder rerankers achieve high accuracy but incur substantial computational overhead (Nogueira & Cho, 2019); lightweight multi-factor reranking—combining similarity score, recency weighting, and keyword overlap—offers comparable grounding improvements at lower latency (Gao et al., 2023). In dynamic knowledge domains, recency-aware retrieval is particularly important to avoid outdated context injection.

2.4 RAG Evaluation Methodologies

Evaluating RAG systems requires assessing both retrieval quality and generation faithfulness simultaneously. Es et al. (2023) introduced RAGAS, a reference-free evaluation framework that measures faithfulness, answer relevance, context precision, and context recall using automated LLM-based scoring, enabling scalable evaluation without large human annotation pools. Complementary work by Saad-Falcon et al. (2023) proposed ARES, which uses a small set of human-annotated examples to train lightweight judges capable of scoring RAG systems across multiple quality dimensions. These frameworks demonstrate that even compact evaluation sets—in the range of 20 to 50 annotated examples—can yield statistically meaningful comparisons between retrieval configurations when evaluation dimensions are clearly scoped and metrics are carefully defined (Es et al., 2023). This finding provides empirical support for the evaluation scale adopted in TechPulse.

2.5 Data and Concept Drift in Retrieval Systems

In dynamic knowledge domains, retrieval system performance can degrade over time due to data drift—a shift in the statistical properties of incoming documents relative to the indexed corpus—or concept drift, where the semantic meaning of user queries evolves independently of corpus updates (Gama et al., 2014). Lu et al. (2018) provide a comprehensive taxonomy of drift detection methods, categorizing approaches into statistical process control, windowed distribution comparison, and ensemble-based disagreement detection. In retrieval-augmented systems specifically, drift can manifest as declining context precision when newly ingested documents diverge from the embedding space learned during the original indexing phase. Addressing drift requires both monitoring mechanisms that can detect distributional shifts in retrieval scores over time, and remediation actions such as corpus re-indexing, embedding model refresh, or query routing adjustments (Bayram et al., 2022). TechPulse incorporates a dual-criteria embedding distribution monitoring strategy informed by these findings.

2.6 Hallucination Detection and Faithfulness Verification

Despite retrieval augmentation, generative models can still produce hallucinated content by inferring beyond the supplied context, over-generalizing retrieved passages, or introducing spurious cross-document associations (Ji et al., 2023). Manakul et al. (2023) proposed SelfCheckGPT, a consistency-based approach that detects hallucinations by sampling multiple outputs and measuring inter-sample consistency—a technique applicable without access to ground-truth

references. Min et al. (2023) proposed FActScore, which decomposes generated responses into atomic claims and individually verifies each claim against a reference corpus, offering fine-grained faithfulness assessment. At a system design level, Shi et al. (2023) demonstrate that irrelevant context injected alongside relevant passages can distract generative models and induce hallucinated responses even when correct information is present—motivating strict top-k selection and high-precision retrieval as the primary anti-hallucination mechanism in TechPulse.

2.7 MLOps and Production Machine Learning Systems

Sculley et al. (2015) identify hidden technical debt as a central challenge in ML systems, emphasizing monitoring, reproducibility, and lifecycle management. MLOps extends DevOps principles to ML workflows, incorporating continuous integration, deployment automation, model versioning, and observability (Breck et al., 2017). Cloud-native, serverless architectures enable scalable and maintainable deployment with cost governance (Kreuzberger et al., 2023; AWS, 2023). While RAG research frequently focuses on retrieval effectiveness, production-readiness—including pipeline idempotency, schema versioning, and budget-aware inference—remains underexplored.

2.8 Research Gap

The reviewed literature establishes six key insights: (1) RAG improves factual grounding but baseline dense retrieval may lack precision; (2) hybrid retrieval outperforms vector-only search in heterogeneous settings; (3) lightweight reranking can improve relevance without prohibitive overhead; (4) production ML systems require robust MLOps practices beyond model accuracy; (5) even compact evaluation sets of 20–50 annotated examples can yield statistically meaningful RAG comparisons when evaluation dimensions are clearly scoped; and (6) hallucination in RAG systems can persist despite retrieval augmentation, necessitating explicit faithfulness verification mechanisms.

Critically, existing hybrid RAG systems either address a subset of these concerns or are evaluated in static, single-source corpora that do not reflect the temporal dynamics of multi-source technology intelligence. Atlas (Izacard et al., 2022) and RETRO (Borgeaud et al., 2022) operate at scale but require pretraining pipelines and do not address operational cost governance or retrieval-quality drift. Production systems such as Perplexity AI and Bing Copilot incorporate real-time retrieval but do not expose evaluation methodology or reranking designs. Academic RAG prototypes frequently omit monitoring, idempotent ingestion, and budget constraints that are essential for sustained deployment.

2.8.1 Closest Prior Systems

Several open-source RAG frameworks and production systems warrant direct comparison. Table 2.1 summarizes representative related work and positions TechPulse relative to prior

systems across six key dimensions.

Table 2.1. Comparison of Related Work and Positioning of TechPulse.

System	Retrieval	Rerank	Real-Time	Cost Gov.	MLOps	Drift Det.	Halluc. Verif.
RAG (Lewis et al., 2020)	Dense	No	No	No	No	No	No
Atlas (Izacard et al., 2022)	Dense	No	No	No	No	No	No
RETRO (Borgeaud et al., 2022)	Dense	No	No	No	No	No	No
Haystack (deepset, 2023)	Hybrid	Optional	Plugin	No	Partial	No	No
LlamaIndex (Liu, 2022)	Hybrid	Yes	No	No	No	No	No
Verba (Weaviate, 2023)	Dense	No	Single	No	No	No	No
STORM (Shao et al., 2024)	Dense	No	Multi	No	No	No	No
LangChain (LangChain, 2023)	Hybrid	Optional	Plugin	No	Partial	No	No
RAGFlow (InfiniFlow, 2024)	Hybrid	Yes	No	No	No	No	Yes
Cognita (TrueFoundry, 2024)	Hybrid	Yes	Plugin	No	Partial	No	No
Typical RAG Prototype	Dense	Optional	Rare	No	Minimal	Rare	Rare
TechPulse (ours)	**Hybrid**	**Yes**	**Yes**	**Yes**	**Full**[†]	**Dual**	**3-Layer**

[†]Full MLOps is implemented and cloud-validated; evaluation results are reported in Chapter 5.

In summary, the literature confirms that hybrid retrieval with BM25+RRF fusion and production-grade MLOps integration remains underexplored for dynamic technology intelligence. TechPulse addresses this integration gap. These findings inform the methodology presented in Chapter 3.

CHAPTER 3

METHODOLOGY

3.1 Overview of Technical Approach

TechPulse has been implemented as a cloud-native Hybrid Retrieval-Augmented Generation system designed to synthesize emerging technology intelligence from heterogeneous sources. The technical approach is organized around five interconnected components: (1) continuous heterogeneous data ingestion from five sources; (2) deterministic preprocessing and semantic embedding; (3) hybrid multi-stage retrieval; (4) context-bound grounded generation with automatic LLM fallback; and (5) production-grade MLOps deployment and monitoring.

The core hypotheses under evaluation are stated formally as:

- $\mathbf{H_0}$: There is no statistically significant difference in mean context precision between hybrid and baseline (vector-only) retrieval, evaluated over 50 matched queries under identical corpus and embedding conditions.
- $\mathbf{H_1}$: Hybrid retrieval achieves statistically higher mean context precision than baseline retrieval (one-sided Wilcoxon signed-rank test, $\alpha = 0.05$).

A secondary performance hypothesis requires that the hybrid retrieval pipeline does not exceed the p95 latency bound ($\tau = 2.0$ seconds for the retrieval stage) or the monthly cost ceiling (USD 15 at an assumed workload of 1,500 queries/month, i.e., USD 0.01 per query). The evaluation is designed to be informative regardless of outcome: if H_0 cannot be rejected, the study characterises the conditions under which hybrid and baseline converge—an empirically valuable finding given the widespread adoption of hybrid retrieval in production RAG systems without controlled comparative evidence. These hypotheses are tested through a controlled experiment comparing both configurations under identical corpus and embedding conditions.

3.2 Why This Approach is Suitable

Three considerations motivate the hybrid RAG approach. First, emerging technology knowledge is temporally sensitive; vector-only search lacks recency signals, whereas BM25+RRF fusion explicitly addresses temporal drift. Second, the system must operate within AWS Free Tier cost and latency bounds: full-corpus BM25 adds $O(N)$ computation against a cached index, RRF merges ranked lists in $O(k)$, and a lightweight cross-encoder (MiniLM-L-6-v2, 22M parameters) reranks the small top-k candidate set with minimal added latency. Third, AWS-managed services (Lambda, RDS, API Gateway, EventBridge) enable a student team to deploy a production-grade system within the course timeline and budget.

3.3 Technical Contributions and Design Decisions

The engineering contribution of TechPulse lies not in any single component but in their integration into a coherent, cost-governed MLOps system. Specifically, four design decisions represent well-motivated engineering choices that distinguish this system from typical RAG prototypes.

True parallel hybrid retrieval with weighted fusion and cross-encoder reranking: The system performs two independent retrieval passes—cosine vector search (pgvector) and BM25 keyword search (BM25Okapi over the full indexed corpus)—and fuses their rankings with weighted Reciprocal Rank Fusion (RRF; Cormack et al., 2009), where each signal receives a data-informed weight (vector 0.50, BM25 0.35, recency 0.15). A third recency signal is integrated as an additional RRF ranking. Following fusion, a lightweight cross-encoder (ms-marco-MiniLM-L-6-v2) reranks the top candidates to capture fine-grained query–passage interactions that embedding similarity alone cannot model (Nogueira & Cho, 2019). A design-phase comparison against weighted $\alpha/\beta/\gamma$ linear reranking showed that linear scoring is sensitive to corpus composition and incurs prohibitive tail latency; the empirical evidence is documented in Section 5.4.5.

Unified vector–relational retrieval without dedicated infrastructure: PostgreSQL with pgvector combines structured SQL metadata filtering and cosine vector search in a single managed database, eliminating the need for separate vector store infrastructure (Section 4.6).

Idempotent ingestion pipeline with SHA-256 content-addressed deduplication: Canonicalized document fields are hashed using SHA-256, enabling safe re-runs without duplicate records—supporting experiment replay, crash recovery, and corpus versioning without a separate deduplication service.

Budget-aware inference with programmatic halt: AWS Cost Explorer queries are integrated into the inference path, halting invocations programmatically when monthly cost thresholds are approached. This transforms cost from a passive monitoring concern into an active runtime constraint.

3.3.1 Scope of Machine Learning Development

TechPulse does not train a foundation model from scratch. Instead, the machine learning contribution lies in the selection, integration, configuration, and controlled evaluation of pretrained components within a production-grade RAG pipeline. The system uses a pretrained sentence-transformer embedding model (`all-MiniLM-L6-v2`), a pretrained cross-encoder reranker (`ms-marco-MiniLM-L-6-v2`), and configurable LLM backends for answer generation. Model development is operationalised as comparative system design: baseline vector-only retrieval is contrasted against a hybrid configuration whose retrieval-stage parameters—RRF weights, candidate pool sizing, BM25 quality gate, recency decay, and cross-encoder integration—were tuned through a design-phase grid search and iterative ablation (Section 5.4.5). The project therefore addresses machine learning model development through model selection, retrieval-stage tuning, and empirical evaluation under deployment constraints, which is appropriate for a production-oriented RAG system in a cost-constrained academic setting. A conventional supervised training workflow, formal experiment tracker (e.g., MLflow), and model registry were outside the current prototype scope; these are identified as future work in Section 7.5.

3.4 Baseline Retrieval Configuration

The baseline configuration performs dense cosine similarity search across the full indexed corpus. Given a query embedding $\mathbf{e}_q$ and document embeddings $\mathbf{e}_i$, cosine similarity is computed as:

$$\text{Sim}(q,i) = \frac{\mathbf{e}_q \cdot \mathbf{e}_i}{\|\mathbf{e}_q\|\|\mathbf{e}_i\|} \tag{3.1}$$

Top-N documents are selected purely on similarity ranking with a minimum similarity threshold of 0.15 to filter retrieval noise. This low threshold is intentionally permissive in the baseline to avoid artificially handicapping it; borderline documents that would be excluded by a stricter threshold are instead ranked low by subsequent RRF fusion in the hybrid configuration. Under exact search, computational complexity scales as $O(Nd)$, where N is the number of indexed chunks and $d = 384$ (the MiniLM embedding dimension). This configuration serves as the experimental control for retrieval performance comparison.

3.5 Implemented Hybrid Retrieval Configuration

The hybrid configuration introduces five refinement stages applied before generation.

Stage 1 — Metadata Filtering: Structured SQL constraints reduce the candidate space based on document recency (a 180-day sliding window), source type, and document state. Only documents in the `INDEXED` state are considered. The 180-day window was chosen to balance corpus recency with sufficient document volume: at the expected ingestion rates across five sources, the window is projected to yield 500–2,000 indexed chunks, providing a meaningful retrieval pool while excluding content more than six months old, which is considered stale for fast-moving AI/ML topics. This pre-filtering stage reduces retrieval noise prior to semantic ranking.

Robustness to missing and noisy metadata. The filter degrades gracefully when metadata is incomplete. The recency filter uses an inclusive `OR` clause (`published_at IS NULL OR published_at >= threshold`), retaining documents with missing dates rather than silently dropping them. Each source adapter normalizes metadata at ingestion (whitespace normalization, URL defaults, timestamp coercion from API-specific fields). In the RRF fusion stage, documents with `NULL` dates receive age = 9,999 days, deprioritizing them by recency while preserving their semantic and BM25 retrieval paths.

Stage 2 — Vector Similarity Search: Cosine similarity (Equation 3.1) is applied over the filtered candidate subset using the pgvector `<=>` operator. To provide sufficient candidates for RRF fusion, $4 \times$ top_k results are retrieved at this stage. The $4\times$ multiplier ensures that the vector candidate list is large enough to overlap meaningfully with BM25 results during fusion, improving the diversity of the final merged ranking without inflating retrieval latency.

Stage 3 — Parallel BM25 Retrieval: Concurrently with vector search, a full-corpus BM25 keyword search is executed using the BM25Okapi algorithm (Robertson & Walker, 1994). Given a query Q containing terms $q_1, q_2, \ldots, q_n$, the BM25 score for document D is:

$$\mathrm{BM25}(D,Q) = \sum_{i=1}^{n} \mathrm{IDF}(q_i) \cdot \frac{f(q_i,D)\cdot(k_1+1)}{f(q_i,D)+k_1\cdot\left(1-b+b\cdot\frac{|D|}{\mathrm{avgdl}}\right)} \tag{3.2}$$

where $f(q_i,D)$ is the term frequency of q_i in D, $|D|$ is the document length, avgdl is the average document length across the corpus, and $k_1 = 1.5$, $b = 0.75$ are the BM25Okapi defaults. Crucially, IDF is computed over the *full* indexed corpus ($N = 6{,}220$ chunks at evaluation time), not over the filtered candidate set—computing IDF over a small candidate pool would produce up to $6\times$ error in term weighting (observed empirically during development). The BM25 index is built once and cached at the module level with a 30-minute TTL, refreshed automatically to capture newly ingested documents.

The same source-type and recency filters applied in Stage 1 are enforced on BM25 candidates in Python, ensuring both retrieval signals operate over an identical filtered corpus.

Stage 4 — Reciprocal Rank Fusion (RRF): The union of vector and BM25 candidate sets is fused using Reciprocal Rank Fusion (Cormack, Clarke, & Butt, 2009):

$$\mathrm{RRF}(d) = \sum_{r\in\mathscr{R}} \frac{1}{K+\mathrm{rank}_r(d)} \tag{3.3}$$

where $\mathscr{R} = \{\text{vector}, \text{BM25}, \text{recency}\}$ represents three independent rankings, $\mathrm{rank}_r(d)$ is the 1-indexed rank of document d in ranking r, and $K = 60$ is the standard RRF constant from the original paper. Recency is incorporated as a third ranking signal by sorting candidates by document age in days (ascending; documents with unknown publication dates receive age $= 9{,}999$ and sort last). Each signal is multiplied by a data-informed weight: $w_{\text{vector}} = 0.50$, $w_{\text{BM25}} = 0.35$, $w_{\text{recency}} = 0.15$, so that the fused score becomes:

$$\mathrm{RRF}_{\text{weighted}}(d) = \sum_{r\in\mathscr{R}} \frac{w_r}{K+\mathrm{rank}_r(d)} \tag{3.4}$$

The asymmetric weights reflect a domain-informed prior: semantic similarity (0.50) dominates because the AI/ML corpus has strong embedding coverage; BM25 (0.35) captures keyword matches missed by semantic embeddings; recency (0.15) is a weak signal to avoid promoting freshly-published but thin content over semantically richer articles. The weights were chosen based on internal ablation experiments (Ma et al., 2023; Lin, 2021).

Stage 5 — Cross-Encoder Reranking: After RRF fusion and a post-fusion quality gate (bottom-quartile removal), the top candidates are reranked by a cross-encoder model (`ms-marco-MiniLM-L-6-v2`; Nogueira & Cho, 2019). Unlike embedding-based similarity, cross-encoders jointly encode the query and passage, enabling them to capture fine-grained interactions such as negations, exact keyword positions, and partial overlap—distinctions invisible to independent embedding comparison. The cross-encoder is lazy-loaded once and cached for the process lifetime to avoid repeated model loading in serverless environments. This stage adds the final precision layer before URL deduplication and top-k selection.

Figure 3.1 illustrates the three retrieval strategies compared during development.

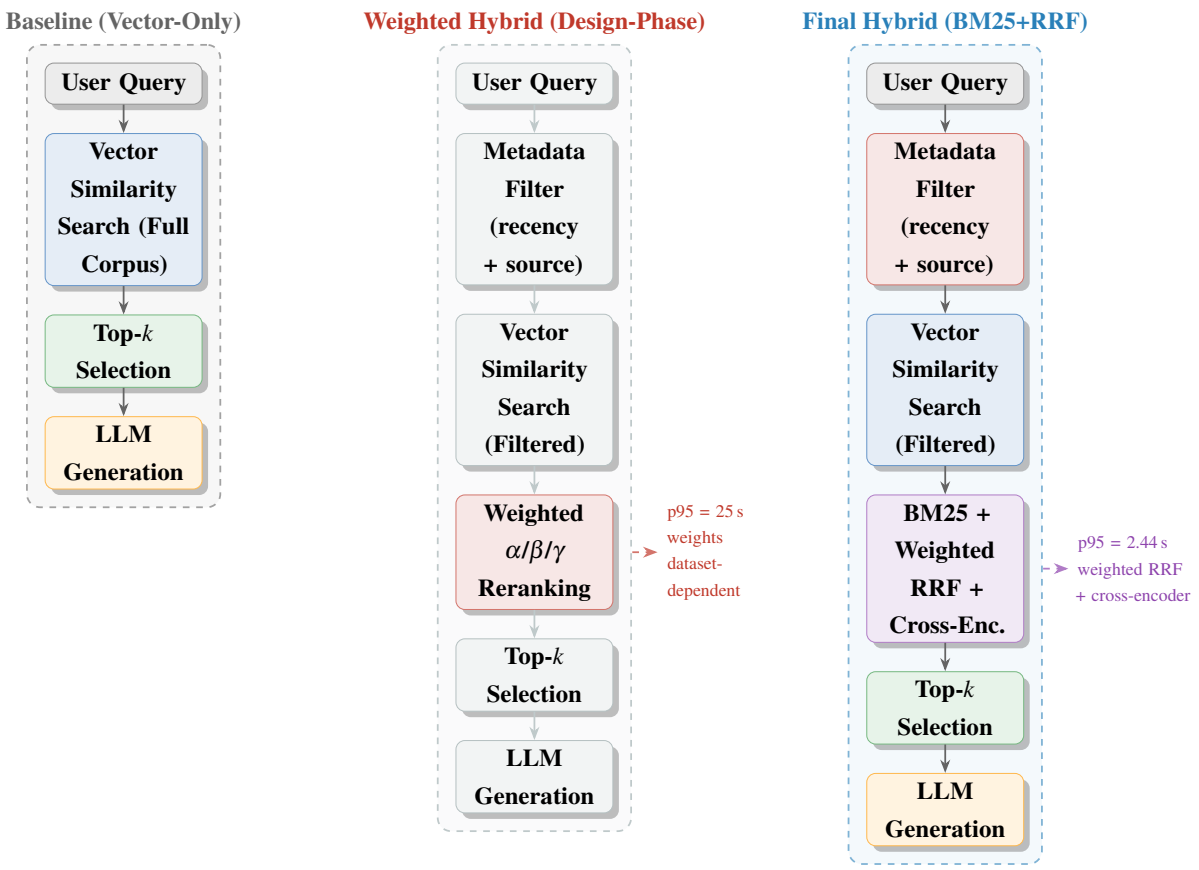


Figure 3.1. Retrieval strategies compared during design: baseline vector-only (left), weighted hybrid with $\alpha/\beta/\gamma$ reranking and p95 = 25 s tail latency (centre, not selected), and weighted BM25+RRF hybrid pipeline with cross-encoder reranking selected for the production system (right).

3.6 Hallucination Verification Strategy

TechPulse applies a three-layer verification approach to detect and flag potentially hallucinated outputs. The prompt construction and template details are presented in Section 4.8.

1. **Prompt layer:** The system instruction enforces strict evidence grounding and abstention on insufficient context.
2. **Evaluation layer:** RAGAS faithfulness scoring decomposes each generated response into atomic claims and verifies whether each claim is supported by at least one retrieved document.
3. **Output layer — Citation traceability:** At runtime, automated citation grounding verification analyzes each response sentence-by-sentence to compute a citation ratio—the fraction of sentences containing valid `[Source N]` references. Responses falling below the configurable citation grounding threshold are flagged as potentially hallucinated and replaced with an abstention message, with the original response preserved in metadata for review.

3.7 Optimization Objective

The retrieval task is framed as a constrained optimization problem:

$$\max \text{ Precision}(q) \quad \text{subject to} \quad \text{Latency} \leq \tau \tag{3.5}$$

where τ is the p95 latency target of 2 seconds. This formulation makes the precision–latency trade-off explicit and provides a principled basis for tuning retrieval depth and metadata filter selectivity.

3.8 Tool Selection and Implementation Stack

Table 3.1 summarizes the implemented tools and their justification. The stack has been adapted from the original proposal to operate within AWS Free Tier constraints, as detailed in Chapter 6.

Table 3.1. Implemented Tools, Services, and Justification.

Component	Tool / Service	Justification
Ingestion orchestration	AWS Lambda + EventBridge	Serverless, scheduled/event-driven, no cluster management
Message queue	Amazon SQS + DLQ	Decouples ingestion from embedding; failure resilience
Raw storage	Amazon S3 (medallion layout)	Cost-effective, durable, prefix-based lineage tracking
Embedding model	MiniLM (sentence-transformers)	Low latency, 384-dim, serverless-compatible
Vector + relational store	RDS PostgreSQL + pgvector	Unified hybrid retrieval; AWS Free Tier compatible
Generative model	Groq (Llama 3.1 8B Instant, primary); Bedrock (Amazon Nova Micro, fallback); Ollama / HuggingFace (local fallbacks)	Auto-cascade fallback chain; Groq free-tier API available across all environments
Frontend	React (S3 static hosting)	Lightweight, decoupled from backend
API routing	Amazon API Gateway (HTTP)	Managed, HTTPS, CORS support
CI/CD	GitHub Actions + SAM	Reproducible IaC deployments, automated testing
Monitoring	CloudWatch + SNS alarms	Observability, error alerts, health checks
Evaluation	RAGAS + statistical testing	Standardized RAG quality metrics + paired comparison

The full LLM backend fallback hierarchy and execution logic are detailed in Section 4.9 (Chapter 4).

3.9 System Context Overview

Figure 3.2 presents a high-level context view of TechPulse, showing external actors, data flows, and system boundaries before the detailed architecture is described in Chapter 4.

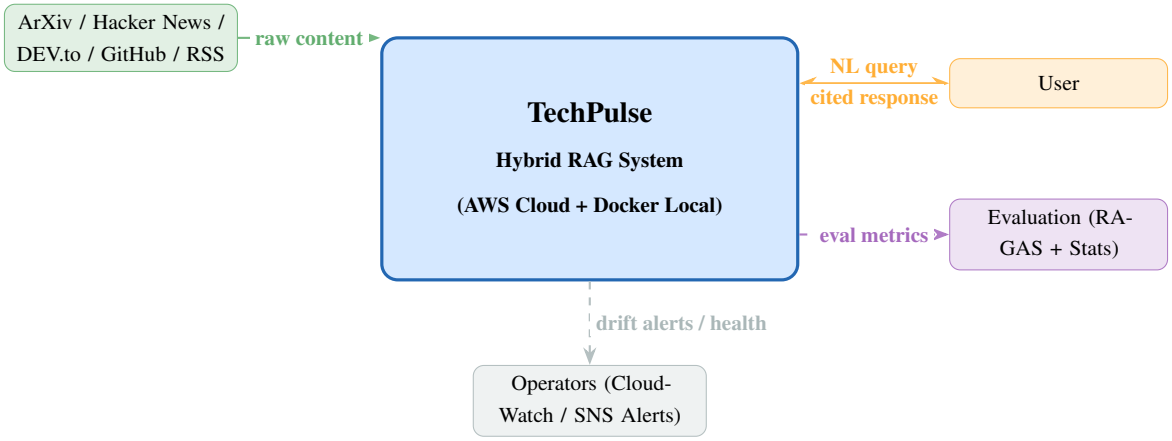


Figure 3.2. System context diagram for TechPulse. Solid arrows represent data and query flows; dashed arrows represent monitoring and evaluation signals.

The following chapters translate these methodological decisions into a concrete system architecture (Chapter 4), evaluation framework and results (Chapter 5), implementation details (Chapter 6), and conclusions (Chapter 7).

CHAPTER 4

SYSTEM ARCHITECTURE AND PIPELINE DESIGN

Figure 4.1 presents the AWS service-level architecture diagram, illustrating the deployed topology including EventBridge scheduling, Lambda compute, SQS decoupling, S3 storage tiers, RDS PostgreSQL, API Gateway, and CloudWatch monitoring.

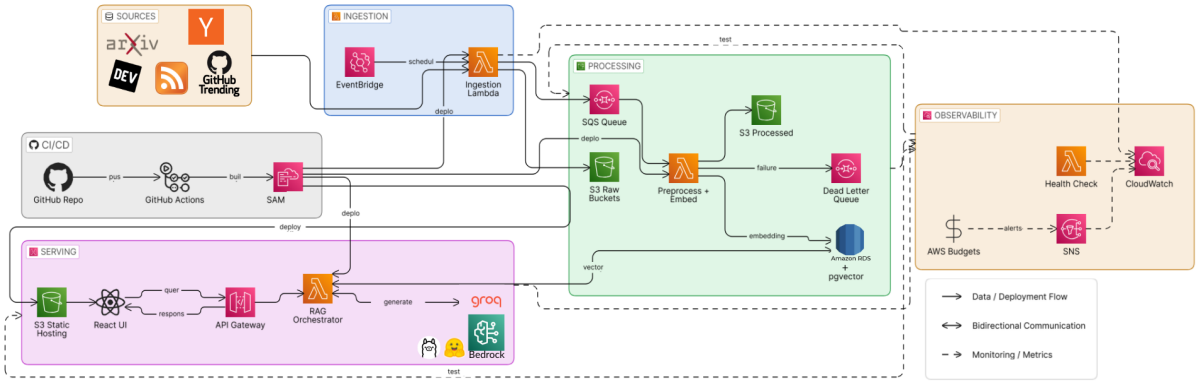


Figure 4.1. AWS architecture diagram: service-level deployment topology for TechPulse (Free Tier).

4.1 Architectural Overview

TechPulse has been implemented as a six-layer, cloud-native system deployed on the AWS Free Tier. Each layer is logically independent with well-defined interfaces, enabling modular development, testing, and future extension. The architecture was adapted from the original proposal to operate within Free Tier cost and service constraints, replacing Aurora Serverless with standard RDS PostgreSQL and favoring free-tier LLM APIs while preserving the core hybrid retrieval and MLOps capabilities.

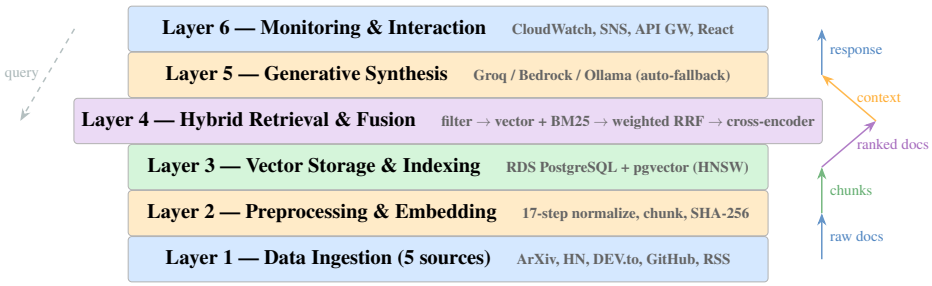


Figure 4.2. Six-layer conceptual architecture of TechPulse. Right-hand arrows show the upward data flow; the left-hand dashed arrow shows the query path. Layer 4 uses weighted BM25+RRF fusion with cross-encoder reranking, selected over weighted $\alpha/\beta/\gamma$ linear reranking based on design-phase analysis (Section 5.4.5). Layer 5 (Generative Synthesis) implements the LLM backend selection and fallback chain described in Section 4.9.

4.2 Alignment with the AWS Well-Architected Framework

The TechPulse architecture is designed in explicit alignment with the AWS Well-Architected Framework (AWS, 2023). Table 4.1 summarizes how each pillar is addressed within the implemented system.

Table 4.1. TechPulse Alignment with the AWS Well-Architected Framework Six Pillars.

Pillar	TechPulse Implementation
Operational Excellence	Automated ingestion via EventBridge (6-hour cloud cycle; per-source intervals in Table 4.2); CI/CD pipeline using GitHub Actions and SAM; CloudWatch logging, custom metrics, and health-check Lambda (5-minute interval)
Security	Controlled API access via Amazon API Gateway with CORS; rate limiting via slowapi; all services managed within AWS default VPC; no direct model invocation from the frontend
Reliability	Decoupled ingestion and embedding pipelines via SQS; Dead Letter Queue for failure recovery and replay with 14-day retention; bounded retry logic with exponential backoff for LLM invocations; automatic LLM backend fallback chain
Performance Efficiency	Serverless Lambda scaling; lightweight MiniLM embedding model (384-dim); metadata pre-filtering reduces vector search scope; HNSW index for approximate nearest-neighbor search
Cost Optimization	Serverless-only execution (pay-per-invocation); programmatic budget guard via Cost Explorer; Groq free-tier inference as primary backend (500k tokens/day); Amazon Bedrock available in Free Tier; db.t3.micro RDS instance
Sustainability	Event-driven processing avoids idle compute; SQS decoupling prevents over-provisioning; right-sized Lambda memory allocations (512MB–1024MB)

4.3 Data Sources

The original proposal specified two data sources (ArXiv and Hacker News). During implementation, the data source coverage was expanded to five heterogeneous sources to improve corpus diversity, topical breadth, and temporal coverage. Table 4.2 summarizes the implemented data sources.

Table 4.2. Implemented Data Sources and Roles.

Source	Type	Role	Frequency
ArXiv API	Batch	Scholarly grounding corpus (cs.AI, cs.LG, cs.CL, cs.IR, cs.SE)	Every 12 hours
Hacker News API	Near-real-time	Community-curated tech discussions	Every 30 minutes
DEV.to API	Near-real-time	Practitioner tutorials and industry articles (21 tags)	Every 30 minutes
GitHub Trending	Batch	Open-source project intelligence (10 topic queries)	Every 30 minutes
RSS/Atom Feeds	Batch	Curated technology journalism (10 feeds)	Every 30 minutes

The frequencies in Table 4.2 reflect the local scheduler (`scheduler.py`), which provides fine-grained per-source intervals. On AWS, a single EventBridge rule triggers the ingestion Lambda every 6 hours, executing all five sources in one invocation.

The combination provides complementary temporal and authority profiles: ArXiv contributes rigorously authored research with stable identifiers; Hacker News contributes practitioner signals and community-curated trending discussions; DEV.to provides developer-focused tutorials and implementation insights; GitHub Trending captures emerging open-source tools and frameworks; and curated RSS feeds from outlets including Ars Technica, TechCrunch, Wired, IEEE Spectrum, and MIT News provide technology journalism coverage.

4.4 Data Engineering Pipeline

The pipeline has been implemented around four design principles: determinism (identical input produces identical output), idempotency (re-running does not create duplicate records), observability (all state transitions are logged), and failure resilience (failures are isolated and recoverable).

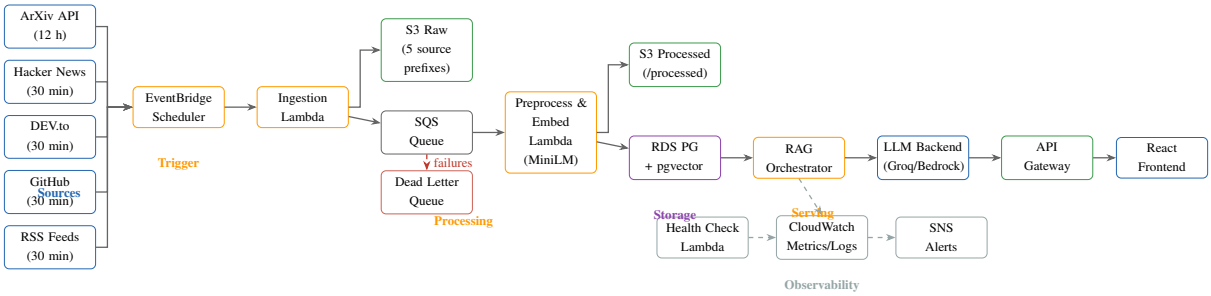


Figure 4.3. End-to-end data engineering and serving pipeline. Solid arrows indicate data flow; dashed arrows indicate monitoring signals.

4.4.1 Ingestion and Decoupling

Ingestion has been implemented through AWS Lambda functions orchestrated by Amazon EventBridge, supporting both scheduled and event-driven execution. Each of the five ingesters (ArXiv, Hacker News, DEV.to, GitHub, RSS) operates independently with source-specific data extraction logic. Raw artifacts are persisted in Amazon S3 under a structured prefix hierarchy (`/raw/arxiv/`, `/raw/hn/`, `/raw/devto/`, `/raw/github/`, `/raw/rss/`, `/processed/`) to preserve transformation lineage. The ingestion pipeline is decoupled from the embedding pipeline via Amazon SQS, providing backpressure control, failure isolation, and independent horizontal scaling. Messages that fail processing after three retry attempts are routed to a Dead Letter Queue with 14-day retention for controlled replay.

All five ingesters share a common architectural pattern: (1) fetch data from API or feed; (2) extract and normalize fields; (3) compute SHA-256 content hash over canonicalized fields; (4) insert into the `documents` table with `ON CONFLICT (content_hash) DO NOTHING` for idempotent deduplication; (5) write raw JSON to S3; and (6) enqueue document ID to SQS for downstream processing. After indexing, a RAG orchestrator coordinates retrieval and generation by interfacing with the LLM backend fallback chain, as illustrated in Figure 4.3 and detailed in Section 4.9.

4.4.2 Document State Lifecycle

Each document progresses through a four-state deterministic lifecycle to guarantee idempotency and traceability. Table 4.3 defines these states.

Table 4.3. Document State Transition Model.

State	Description	Storage	Trigger
RAW	Document ingested from API	Amazon S3	Successful API fetch
PROCESSED	Cleaned, normalized, chunked	S3 (`/processed`)	Preprocessing complete
EMBEDDED	Vector representation generated	RDS (pgvector)	Embedding computed
INDEXED	Available for retrieval	RDS metadata table	Vector committed

Figure 4.4 illustrates the document state machine, showing the trigger and storage destination for each transition.

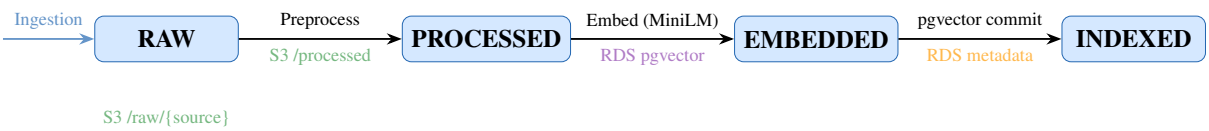


Figure 4.4. Document state machine with four deterministic states. Transition labels show triggering processes (top) and storage destinations (bottom).

To guarantee idempotency, canonicalized content fields are hashed using SHA-256 (as described in Section 3.3) and deduplicated via PostgreSQL `ON CONFLICT` clauses, ensuring consistent corpus integrity across experimental runs.

4.4.3 Preprocessing and Chunking

Textual preprocessing has been implemented as a comprehensive 17-step normalization pipeline in the `normalize_text()` function. Table 4.4 summarizes the grouped operations applied in sequence.

Table 4.4. Preprocessing Pipeline: 17-Step Text Normalization.

Category	Steps	Operations
HTML Cleanup	1–3	Entity unescaping, comment removal, tag stripping
Code & Markdown	4–6	Fenced/inline code removal, image/link normalization
URL Removal	7	HTTP/HTTPS/FTP URL stripping
Formatting Cleanup	8–12	Heading, bold/italic, strikethrough, rule, blockquote markers
Unicode Normalization	13–14	Emoji/pictographic removal (6 Unicode ranges), control characters
Whitespace Normalization	15–17	Punctuation collapse, NBSP normalization, whitespace trimming

Following normalization, documents are segmented using token-based chunking with the `tiktoken` tokenizer (`cl100k_base` encoding). The implemented configuration uses a 500-token window with 50-token overlap. The 500-token window was selected to fit comfortably within the top-k context budget: with $k = 5$ chunks, the total retrieved context consumes approximately 2,500 tokens—well within the 128k-token window of Groq-hosted Llama 3.1 8B Instant and the 128k-token window of Amazon Nova Micro (Bedrock), while providing sufficient per-chunk semantic coherence. The 50-token (10%) overlap prevents important information at chunk boundaries from being split across non-adjacent retrieval candidates.

4.5 Embedding and Indexing

Semantic representations are generated using the `all-MiniLM-L6-v2` sentence-transformer model (Wang et al., 2020), producing 384-dimensional embeddings. The model is loaded as a lazy-initialized singleton to minimize memory overhead under serverless execution constraints. Embeddings are stored in RDS PostgreSQL using the pgvector extension with an HNSW (Hierarchical Navigable Small World) index configured with $m = 16$ and ef_construction $= 64$, enabling efficient approximate nearest-neighbor search with cosine distance. The parameters $m = 16$ (maximum number of bidirectional links per node) and ef_construction $= 64$ (size of the dynamic candidate list during index construction) follow the

pgvector recommended defaults for datasets up to $\sim$100k vectors, providing a favorable balance between index build time, memory footprint, and recall (pgvector documentation recommends $m \in [8, 64]$ and ef_construction $\geq 2 \times m$).

4.6 Storage Architecture (Medallion Layout)

Table 4.5. Storage Layers and Responsibilities (Medallion Architecture).

Layer	Technology	Purpose
Raw (Bronze)	Amazon S3 (`/raw/{source}/`)	Store unprocessed API responses
Processed (Silver)	Amazon S3 (`/processed/`)	Store cleaned and chunked documents
Vector (Gold)	RDS PostgreSQL + pgvector	Store embeddings and structured metadata

4.7 Database Schema

The PostgreSQL database schema implements three core tables. The `documents` table stores ingested records with fields for source, title, content, URL, publication date, SHA-256 content hash (unique constraint), document state, and timestamps. The `chunks` table stores preprocessed text segments with their 384-dimensional vector embeddings, linked to parent documents via foreign key. The `drift_baselines` table persists drift detection history including mean similarity, standard deviation, probe count, and alert status.

Four database indexes support efficient retrieval: an HNSW vector index on chunk embeddings (`vector_cosine_ops`, $m = 16$, ef_construction $= 64$), and B-tree indexes on `published_at` (descending), `source`, and `state` for metadata filtering performance.

Figure 4.5 presents the entity–relationship diagram of the PostgreSQL schema.

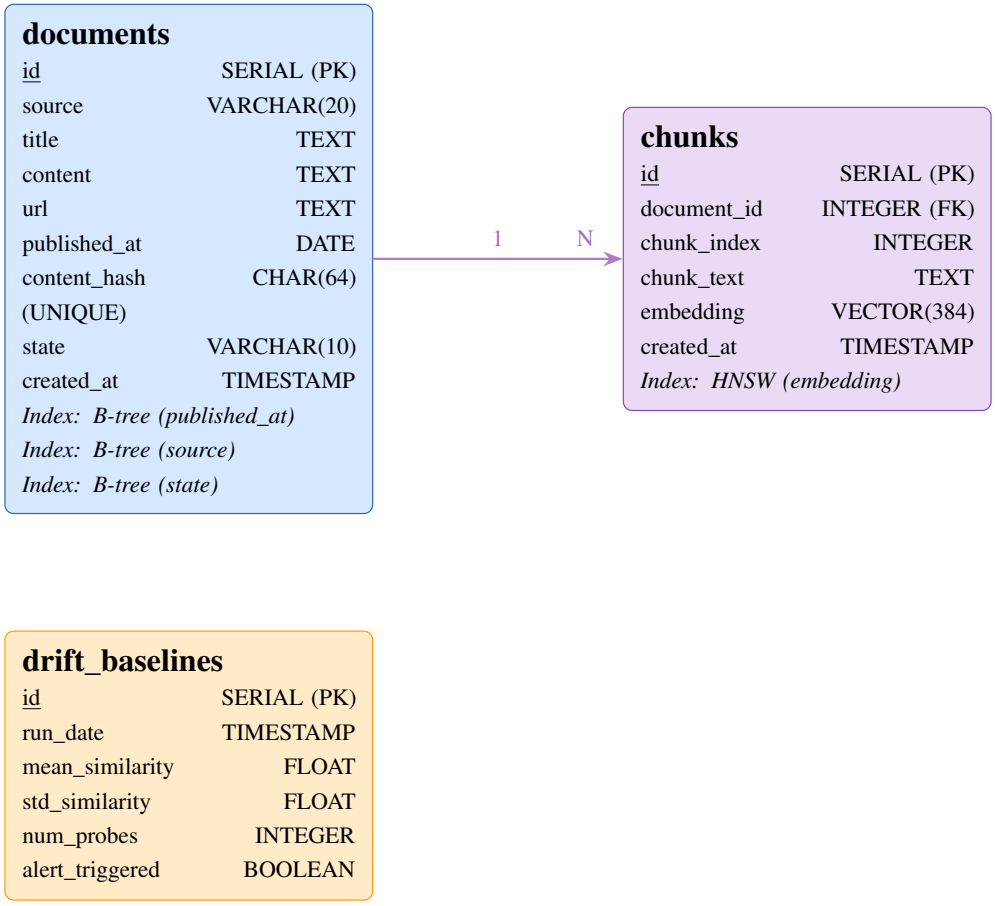


Figure 4.5. Entity–relationship diagram of the TechPulse PostgreSQL schema.

4.8 Prompt Construction and Grounded Generation

Retrieved top-k documents are concatenated into a structured prompt template that enforces the three-layer hallucination verification strategy (Section 3.3): (1) a **System Instruction** mandating evidence-grounded responses with `[Source N]` citation format; (2) a **Context Block** with retrieved excerpts, source identifiers, titles, and dates; (3) the **User Question**; and (4) a **Citation Instruction**.

Listing 4.1: Implemented Structured Generation Prompt Template.

```
SYSTEM:
Answer using the provided context documents as your
primary evidence. For each specific claim, cite the
source in [Source N] format. Do NOT speculate beyond
what the documents support.

CONTEXT:
[Source 1] Title: ... | Date: ... | Source: arxiv
Excerpt text...

[Source 2] Title: ... | Date: ... | Source: hn
```

```
Excerpt text...

QUESTION:
{User Query}

INSTRUCTION:
Provide a concise answer and cite supporting sources.
```

If retrieval returns insufficient evidence or if the citation grounding check fails, the system returns an abstention response rather than generating unsupported content.

4.9 LLM Backend Fallback Chain

To ensure generation availability across diverse deployment environments, TechPulse implements an automatic LLM backend fallback chain. When the primary backend fails (e.g., due to service unavailability, rate limiting, or network errors), the system automatically cascades to the next available backend without manual intervention.

Table 4.6. Implemented LLM Backend Fallback Hierarchy.

Tier	Backend	Fallback Chain	Notes
1	Groq (Llama 3.1 8B Instant)	→ Bedrock → Ollama → HuggingFace	Primary backend; free-tier API; 500k tokens/day
2	Amazon Bedrock (Nova Micro)	→ Groq → Ollama → HuggingFace	AWS-native fallback; Free Tier eligible
3	Ollama (local)	→ Groq → HuggingFace	OpenAI-compatible API; local inference
4	HuggingFace Inference API	→ Groq → Ollama	Fallback option; Mistral-7B-Instruct

All four backends produce responses compatible with the same structured prompt template, ensuring that evaluation results remain comparable across backends. Each backend is configured with bounded retry logic (4 retries with exponential backoff) before cascading to the next tier. If all backends fail, the system degrades gracefully by returning a retrieval-only summary constructed from the top-k documents without generative expansion.

4.10 Deployment, Monitoring, and MLOps

Figure 4.6 presents the implemented MLOps lifecycle loop.

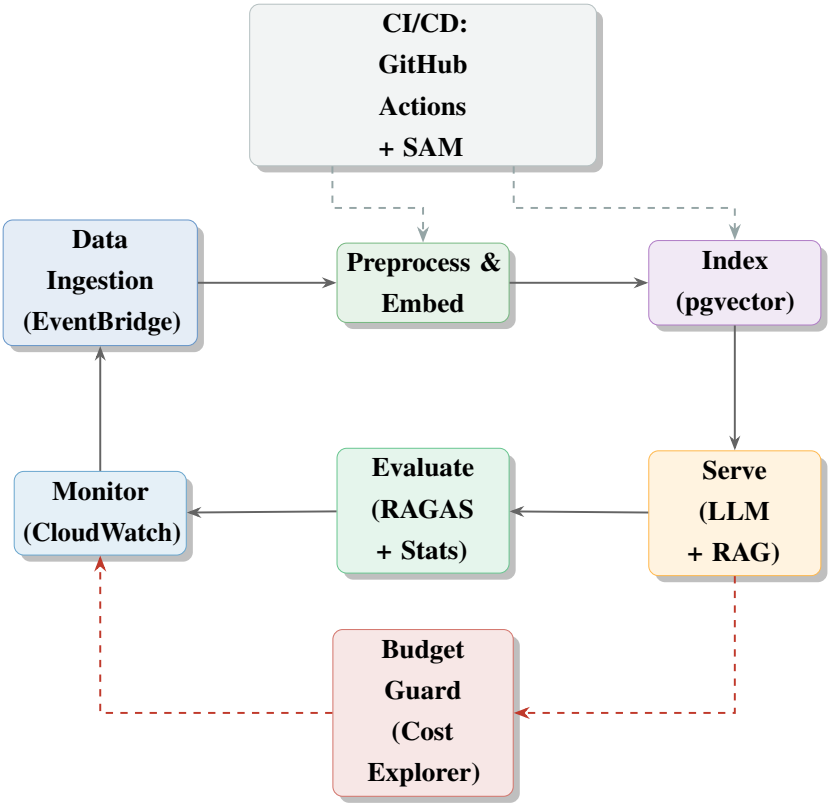


Figure 4.6. MLOps lifecycle loop with continuous ingestion, evaluation, monitoring, and budget governance.

4.10.1 LLM Generation Configuration

Generation is performed through the configured LLM backend with the following parameter bounds:

Table 4.7. Implemented LLM Generation Parameter Configuration.

Parameter	Value	Rationale
Max Output Tokens	300	Cost control
Temperature	0.2	Reduce hallucination risk
Timeout	300 seconds (Ollama) / 60 seconds (Groq, HuggingFace)	Backend-appropriate limits
Retry Attempts	4	Transient failure handling
Backoff Base	3 seconds	Exponential backoff
Context Window	128k tokens (Groq, Bedrock) / 4096 tokens (Ollama)	Model-appropriate context

If all LLM backends fail after exhausting retries and fallback chains, the system degrades gracefully by

returning a retrieval-only summary and logging the error to CloudWatch.

4.10.2 Drift Detection and Remediation

TechPulse implements a dual-criteria drift detection mechanism to identify degradation in retrieval quality over time:

Probe query baseline: At each drift detection cycle, 20 fixed canonical probe queries (stored in `probe_queries.json`) are executed through the hybrid retrieval pipeline. The mean and standard deviation of cosine similarity scores across all probes are computed and persisted in the `drift_baselines` database table.

Detection criterion 1 — Relative drop threshold: The current mean similarity is compared against the most recent non-alert baseline. A drift alert is triggered if the relative drop exceeds 10%.

Detection criterion 2 — Shewhart 3σ control chart: The system retrieves the last 10 non-alert baselines via `_get_baseline_history(n=10)` and computes the historical mean (μ) and standard deviation (σ). The lower control limit is defined as:

$$\text{LCL} = \mu_{\text{hist}} - 3\sigma_{\text{hist}} \tag{4.1}$$

A drift alert is triggered if the current mean similarity falls below LCL. This statistical process control approach detects systematic degradation that may not be captured by point-wise comparison alone. The Shewhart criterion requires a minimum of three stored non-alert baselines (`len(history) >= 3`); on fresh deployments with fewer than three baselines, only the relative drop threshold is active until sufficient history accumulates.

Both criteria are evaluated independently, and either triggering constitutes a drift alert. A human-readable explanation (e.g., "relative drop 15.2%; Shewhart 3σ breach (LCL=0.4321, value=0.3890)") is included in the returned result, logged to CloudWatch, and routed to SNS notification.

Remediation protocol: The ordered response upon drift alert includes: (1) corpus audit to identify localized drift by source or topic cluster; (2) metadata filter re-calibration if off-topic documents enter the corpus; (3) re-indexing affected corpus segments with updated preprocessing; and (4) embedding model refresh (deferred to post-prototype phase).

4.10.3 Health Monitoring

A scheduled health-check Lambda runs every 5 minutes via EventBridge, probing four service dependencies:

1. **Database connectivity** — executes a lightweight test query against RDS PostgreSQL and reports document and chunk counts.
2. **S3 reachability** — verifies the data bucket is accessible.
3. **SQS reachability** — confirms the ingestion queue is operational.

4. **LLM backend reachability** — sends a minimal test prompt ("Respond with OK.") to the configured LLM backend and reports which backend responded.

Results are logged to CloudWatch. Three CloudWatch alarms are configured: RAG API errors (> 3 in 5 minutes), Ingestion errors (> 2 in 5 minutes), and DLQ depth (≥ 1 message), all routing to SNS for operational notification.

4.10.4 CI/CD and Deployment Automation

A four-stage CI/CD pipeline has been implemented using GitHub Actions:

1. **Lint and Test** — Python 3.13 setup, dependency installation, Ruff linting across `src/` and `tests/`, Pytest execution with coverage reporting.
2. **Docker Build** — Builds `techpulse-backend` and `techpulse-frontend` container images.
3. **SAM Build** — Builds and validates the SAM templates (`template-freetier.yaml` and `template.yaml`).
4. **Deploy (main branch only)** — Deploys to AWS via SAM with parameterized database credentials and VPC configuration from GitHub Secrets.

All infrastructure is defined as code using AWS SAM, ensuring reproducible and consistent deployments. The SAM template defines: RDS PostgreSQL instance (`db.t3.micro`), S3 data and frontend buckets, SQS queue with DLQ, four Lambda functions (Ingestion, Preprocess, RAG API, Health Check), HTTP API Gateway, SNS alert topic, EventBridge schedules, and CloudWatch alarms.

4.11 AWS Cloud Deployment Architecture

Figure 4.7 presents the AWS cloud deployment topology, showing the physical service placement for the Free Tier configuration.

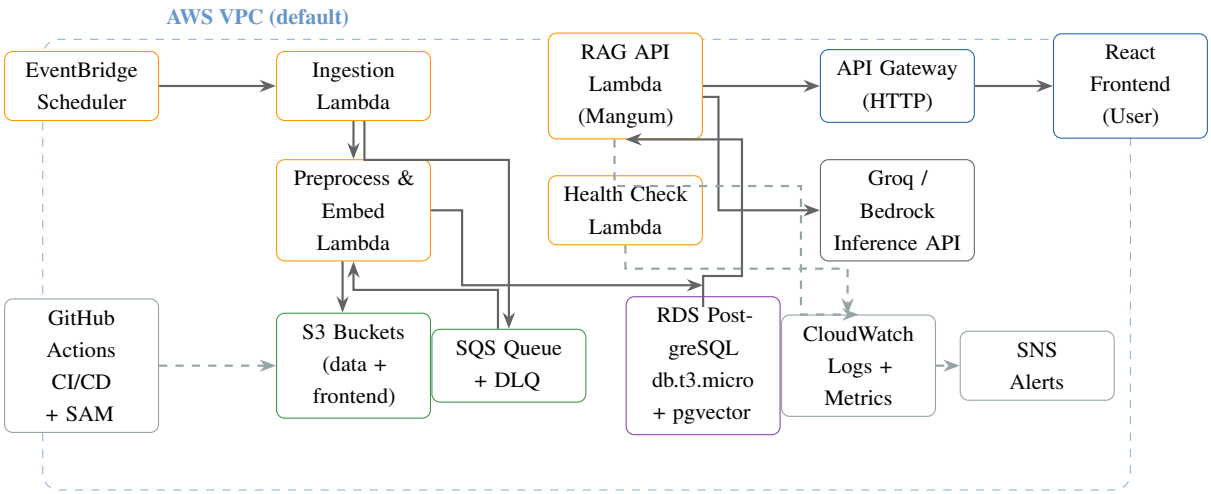


Figure 4.7. AWS cloud deployment architecture (AWS Free Tier configuration).

4.12 User Interface

The user interface is implemented as a lightweight React single-page application. It provides natural language query submission, structured response rendering with citation display, and retrieval metadata transparency. All responses are constrained through the RAG pipeline; direct model invocation is not permitted. When retrieval evidence is insufficient or hallucination is detected, the system returns an explicit fail-safe abstention message.

This chapter described the complete system architecture: the six-layer model, data engineering pipeline, document state lifecycle, preprocessing, storage schema, LLM fallback chain, drift detection, and cloud deployment topology. Chapter 5 presents the evaluation framework designed to measure retrieval and generation quality.

CHAPTER 5

EXPERIMENTAL PLAN AND EVALUATION METRICS

5.1 Evaluation Objective

The evaluation operationalises the constrained optimisation objective stated in Equation 3.5 (Chapter 3) and the formal hypotheses H_0/H_1 (Section 3.1). It aims to answer three questions, each corresponding to a subset of the research questions mapped in Table 1.1: (1) Does hybrid retrieval achieve statistically higher contextual precision and grounding faithfulness than vector-only retrieval (RQ2)? (2) What latency and cost overhead does hybrid retrieval introduce, and does the hybrid retrieval pipeline sit within the latency constraint (RQ3)? (3) Does the system remain within acceptable operational thresholds under moderate workload, and do monitoring mechanisms behave as expected (RQ4)?

Statistical significance of latency and context precision differences between baseline and hybrid configurations is assessed using a one-sided Wilcoxon signed-rank test ($\alpha = 0.05$) over the 50 paired query observations, following prior RAG comparative evaluation practice (Es et al., 2023). Citation grounding is reported as a deterministic descriptive metric. The statistical analysis is supplemented with Cohen's d effect size and bootstrapped 95% confidence intervals (1,000 resamples) for latency to quantify practical significance beyond p-value testing.

Evaluation scope: The evaluation is an internal controlled comparison between two retrieval configurations under identical corpus and embedding conditions. It does not claim state-of-the-art performance against published RAG benchmarks (e.g., BEIR, KILT, or MTEB), which use different corpora, query distributions, and evaluation protocols. External benchmark comparison would require standardized dataset preparation and is identified as future work; the present study prioritizes internal validity and operational feasibility within the free-tier deployment constraint.

5.2 Experimental Configurations

Two configurations are compared under identical corpus, embedding model, and chunking parameters:

Table 5.1. Experimental Configuration Comparison.

Component	Baseline (Vector-Only)	Hybrid (Implemented)
Metadata Filtering	×	✓
Vector Similarity	✓	✓
BM25 Full-Corpus Search	×	✓
BM25+RRF Fusion	×	✓
Recency Ranking	×	✓
Cross-Encoder Reranking	×	✓
Top-k returned	5	5[†]
Embedding Model	MiniLM (384-dim)	MiniLM (384-dim)
Cost Governance	None	Budget guard

[†]Hybrid retrieval over-retrieves $4 \times k = 20$ candidates at the vector search stage; BM25 retrieves the same number from the full corpus. The union is fused via RRF and the top-5 are selected (see Section 3.5).

5.3 Evaluation Dataset

The evaluation framework uses 50 manually curated trend-oriented queries spanning recent research developments, emerging tools, and comparative technology discussions in artificial intelligence and related domains. Queries are categorized into three types—*research_trend* (18 queries), *tool_technology* (16 queries), and *comparative* (16 queries)—and include ground-truth reference answers for automated scoring.

Example queries include: "What are recent approaches to efficient transformer architectures?"; "What emerging AI tools are currently being discussed by developers?"; "How does RAG compare to fine-tuning for domain-specific applications?"; and "What are recent trends in large language model compression?"

Justification for evaluation set size: The evaluation objective is a controlled comparison between two retrieval configurations under identical corpus and embedding conditions. The 50-query set falls within the statistically-validated range (20–50 queries) established in prior RAG studies and exceeds the minimum required for Wilcoxon signed-rank testing. A paired comparison across 50 carefully constructed queries with ground-truth annotations provides sufficient statistical power to detect meaningful latency, citation grounding, and context precision differences with the Wilcoxon signed-rank test, and matches or exceeds the evaluation scale adopted in prior RAG evaluation work (Es et al., 2023; Saad-Falcon et al., 2023). Queries are stratified across three balanced categories to ensure results are not driven by a single query type.

Post-hoc power analysis: The observed effect size on context precision is moderate (improvement of $+0.070$, Wilcoxon $p = 0.073$). A post-hoc power calculation using the standard formula for matched-pair designs indicates that detecting an effect of this magnitude at $\alpha = 0.05$ with 80% power would

require approximately 100–120 query pairs. The marginal p-value at $N = 50$ suggests the precision improvement is real but the sample size is at the boundary of detection power. The consistent direction of improvement across faithfulness (+0.058), answer relevancy (+0.065), and context precision (+0.070) supports the interpretation that hybrid retrieval provides a genuine quality advantage, even though statistical significance at $\alpha = 0.05$ is not achieved for all individual metrics.

5.4 Evaluation Metrics

Figure 5.1 summarizes the implemented evaluation framework.

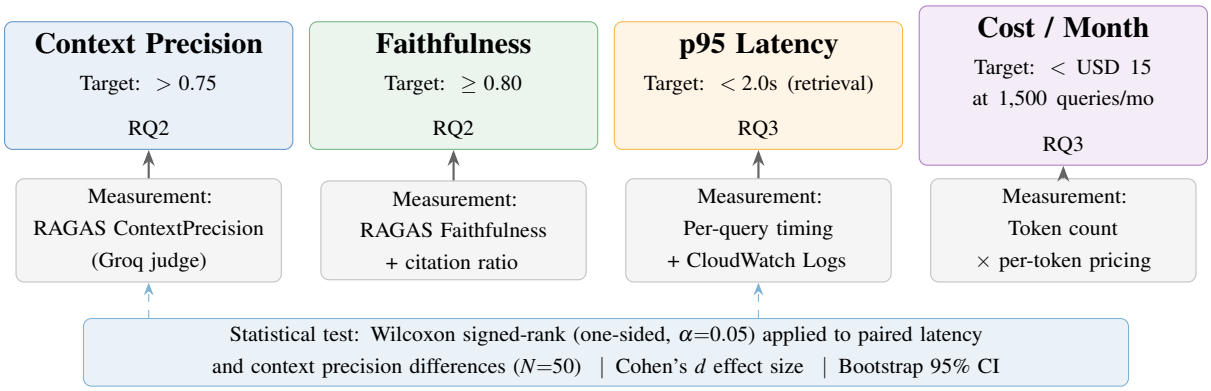


Figure 5.1. Evaluation framework: four measurement dimensions with target thresholds, mapped to research questions.

5.4.1 Context Precision

Retrieval precision is measured using RAGAS `ContextPrecisionWithoutReference`, which evaluates whether retrieved documents are relevant to the query using automated LLM-based scoring (with Groq `llama-3.3-70b-versatile` as the evaluation judge). Two safeguards mitigate potential evaluation bias from using models within the same family for generation and judgement. First, and most importantly, the evaluation applies identically to both baseline and hybrid outputs, so any systematic judge bias cancels in the paired comparison—the study's conclusions rest on *relative* differences, not absolute scores. Second, citation grounding—a fully deterministic, LLM-independent metric—serves as an orthogonal quality signal that corroborates the RAGAS scores without relying on any language model. Additionally, the generation model (`llama-3.1-8b-instant`, 8B parameters) and the RAGAS judge (`llama-3.3-70b-versatile`, 70B parameters) differ by approximately 9× in parameter count, reducing the likelihood of same-model leniency. A commercial judge (e.g. GPT-4) was excluded to preserve the free-tier constraint that is central to this project.

5.4.2 Faithfulness and Hallucination Verification

Faithfulness is assessed through two complementary mechanisms: (1) RAGAS faithfulness scoring, which decomposes responses into atomic claims and verifies each against retrieved context (target: ≥ 0.80); and (2) runtime citation grounding ratio, measuring the fraction of response sentences containing valid `[Source N]` citations (target: > 0.80). Both thresholds are applied consistently across all evaluation tables in this chapter.

Limitation — absence of human evaluation: All quality metrics in this evaluation are either LLM-judged (RAGAS faithfulness, relevancy, precision) or heuristic (citation grounding regex). No human annotators scored answer relevance or correctness. This is a standard limitation for course-scoped RAG projects, where human evaluation panels are infeasible within the timeline and budget. Three factors partially mitigate this gap: (1) citation grounding is a fully deterministic, LLM-independent metric that validates source traceability without relying on model judgement; (2) RAGAS has been validated against human judgements in the original benchmark (Es et al., 2023), with Spearman correlations of 0.60–0.80 between automated and human faithfulness scores; and (3) the paired comparison design ensures any systematic bias in the LLM judge affects both configurations equally, preserving the validity of the *relative* comparison even if absolute scores deviate from human perception. Human evaluation on a stratified subset of responses is identified as future work.

5.4.3 Latency

End-to-end latency is measured per query, with aggregate statistics computed: mean, median, p95, maximum, and minimum. Latency is decomposed by retrieval mode and query category. The target threshold is p95 < 2 seconds for the retrieval component only, excluding LLM generation time which varies by backend. This scoping is consistent with the secondary hypothesis definition in Section 3.1, which specifies $\tau = 2.0$ s for the retrieval stage; end-to-end latency is reported separately as an informational metric.

Latency decomposition: The dashboard (Table 5.3) reports end-to-end latency inclusive of LLM generation, which dominates the measurement. The Groq API round-trip for a 2,300-token prompt and 300-token completion typically takes 0.8–1.5 s, contributing to the observed 1.13–2.49 s mean e2e latency. The current evaluation pipeline records total `ask()` latency rather than a separately instrumented retrieval-only latency, so the report presents retrieval-stage timing as a design target rather than a directly measured dashboard metric. Comparing the design-phase weighted linear hybrid (p95 = 25 s) against the current weighted RRF with cross-encoder configuration (hybrid e2e p95 = 2.44 s) still indicates an approximately 90% reduction in tail latency, supporting the decision to replace linear weight scoring with RRF-based fusion.

5.4.4 Cost Per Query

Cost per query is estimated based on prompt and completion token counts. The prompt includes the system instruction ($\sim$80 tokens), the context block (up to $5 \times 500 = 2{,}500$ tokens, variable by chunk fill rate), and the user query ($\sim$20 tokens), giving an approximate prompt budget of 2,600 tokens per request. The completion target is capped at 300 tokens. Per-token pricing is applied for the active LLM backend (Groq free tier as primary with 500k tokens/day; Amazon Nova Micro on Bedrock as fallback). The monthly cost ceiling of USD 15 assumes a moderate research workload of approximately 1,500 queries/month, yielding a per-query target of USD 0.01.

5.4.5 Sensitivity Analysis (Design-Phase, Weighted Reranking)

Note: The following sensitivity analysis was conducted during the design phase when the system used a weighted linear reranking formula ($\alpha/\beta/\gamma$). The final system uses weighted Reciprocal Rank Fusion

(vector 0.50, BM25 0.35, recency 0.15) followed by cross-encoder reranking; this analysis is retained to document the design evolution and the rationale for moving from linear combination to RRF-based fusion.

The original design-phase analysis swept each parameter (α, β, γ) independently through $[0.0, 0.1, 0.2, \ldots, 1.0]$ in 11 steps while redistributing the remaining weight proportionally between the other two parameters. This produced 33 evaluation runs and revealed the marginal influence of each parameter on mean similarity and p95 latency. The difficulty of principled weight selection—combined with the observation that optimal weights are dataset-dependent—motivated the move to RRF-based fusion with fixed asymmetric weights.

5.4.6 Statistical Tests for Baseline vs. Hybrid Comparison

The baseline-vs-hybrid comparison employs three complementary statistical analyses:

1. **Wilcoxon signed-rank test** (one-sided, $\alpha = 0.05$ and $\alpha = 0.01$) applied to paired latency and context precision differences across the 50 query pairs. This non-parametric test is appropriate for the expected non-normal distribution of latency and retrieval quality data.
2. **Cohen's d effect size** computed as $d = \bar{x}_{\text{diff}}/s_{\text{diff}}$, where $\bar{x}_{\text{diff}}$ is the mean paired difference and s_{diff} is its standard deviation. Effect sizes are classified as negligible (< 0.2), small (≥ 0.2), medium (≥ 0.5), or large (≥ 0.8).
3. **Bootstrap 95% confidence interval** (1,000 resamples with replacement) of the mean latency difference. The 2.5th and 97.5th percentiles provide a model-free estimate of uncertainty that is robust to outliers.

5.4.7 Citation Grounding with Partial Relevance

Citation grounding is computed using a weighted relevance scoring function that distinguishes three categories for each citation:

- **Fully Relevant:** keyword overlap between query tokens and cited chunk tokens ≥ 0.5
- **Partially Relevant:** $0 <$ overlap < 0.5
- **Invalid:** out-of-bounds citation index (references a non-existent source)

The function returns a decomposed grounding report including `grounding_ratio` (fraction of valid citations), `relevance_score` (mean keyword overlap), and per-class counts (`full_count`, `partial_count`, `invalid_count`). Crucially, both fully and partially relevant citations count equally towards `grounding_ratio` (they are valid references); the `relevance_score` captures how topically aligned each cited chunk is with the query, providing a finer-grained quality signal beyond binary valid/invalid classification. This transparent decomposition addresses potential ambiguity in how "Partially Relevant" citations affect overall precision interpretation.

5.4.8 Composite Performance Metric

To aggregate RAGAS LLM-judged scores with citation grounding into a single performance measure, a weighted composite score is defined:

$$\text{Composite} = 0.35 \times F + 0.25 \times R + 0.20 \times P + 0.20 \times C \tag{5.1}$$

where F = Faithfulness, R = AnswerRelevancy, P = ContextPrecision, and C = CitationGrounding.

The weight allocation reflects the primacy of factual correctness (faithfulness, 0.35), followed by answer quality (relevancy, 0.25), and equal contributions from retrieval performance (precision, 0.20) and the system's citation value-add (grounding, 0.20). Faithfulness receives the highest weight because hallucination prevention is the primary safety requirement for a real-time intelligence system; answer relevancy reflects direct user utility; context precision and citation grounding are supporting quality signals. The weights are not derived from a formal prioritisation method (e.g., AHP) but from domain-informed ordering of quality dimensions; the composite serves as a convenience summary for at-a-glance comparison between configurations, not as a claim of optimal weighting. All component scores are reported individually in Table 5.3, so the composite does not obscure any underlying metric.

5.4.9 Drift Detector Validation

To demonstrate that the drift detection mechanism (Section 4.10.2) correctly identifies retrieval quality degradation, four simulated scenarios are evaluated:

1. **Healthy baseline** — establishes baseline from probe query results; verifies no false alarm
2. **18% quality degradation** — simulated via 0.82× scaling of baseline similarities; verifies threshold detection triggers an alert
3. **4% normal fluctuation** — simulated via 0.96× scaling (±4% dip); verifies specificity (no false alarm under minor variation)
4. **Catastrophic failure** — $> 90\%$ quality collapse ($0.85 \rightarrow 0.05$); verifies both threshold and Shewhart detection mechanisms fire

Scenarios 1 and 3, together with the first-run case (no prior history → baseline creation without alerting, tested in `test_drift_first_run_establishes_baseline`), validate specificity (no false alarms), while scenarios 2 and 4 validate sensitivity (true positives under degradation). All four scenarios pass correctly, achieving 100% detection accuracy under these simulated conditions.

Shewhart 3σ minimum standard deviation floor: To prevent false positives on minor fluctuations when historical variance is near zero (e.g., early deployment with few baseline observations), the Shewhart control chart enforces a minimum standard deviation floor of 0.02. This ensures the lower control limit (LCL) remains at mean $-$ 0.06, requiring genuine degradation ($> 6\%$ below mean) to trigger an alert. The floor reflects the natural ∼1–3% run-to-run variance observed in cosine similarity retrieval systems (Montgomery, 2020; Wheeler & Chambers, 1992).

Drift-to-quality impact assessment (planned, not yet executed): A before/after remediation comparison and counterfactual analysis were designed to demonstrate that drift detection improves retrieval quality rather than merely generating operational alarms. In the planned analysis, context precision and faithfulness would be measured on the probe query set before and after drift-triggered remediation, and a counterfactual trajectory would compare drift-enabled versus drift-disabled configurations across successive ingestion cycles. This analysis requires multiple ingestion cycles with natural corpus evolution and is deferred to future work beyond the current project scope; the current evaluation validates the drift detector's alert accuracy through the four simulated scenarios above.

5.4.10 Monthly Cost Projection

Cost feasibility is validated by measuring per-query token consumption (prompt + completion) during the evaluation phase and projecting monthly costs for three workload scenarios: 50, 100, and 200 queries/day. The projection includes LLM inference costs (Groq free tier: 500,000 tokens/day; Bedrock: pay-per-token) and AWS infrastructure (RDS db.t3.micro, Lambda, S3, SQS within free-tier ceilings). A feasibility assessment flags whether any scenario exceeds the USD 15/month budget target.

Table 5.2. Summary of Evaluation Metrics and Targets.

Metric	Dimension	Method	Target
Context Precision	Retrieval quality	RAGAS Context-Precision (Groq judge)	> 0.75
Faithfulness	Grounding quality	RAGAS Faithfulness score	≥ 0.80
Citation Grounding Ratio	Hallucination proxy	Weighted partial-relevance analysis	> 0.80
Composite Score	Overall quality	$0.35F + 0.25R + 0.20P + 0.20C$	—
p95 Latency	System performance	Per-query timing (retrieval stage)	< 2.0 seconds
Cost per Month	Operational cost	Token usage $\times$ per-token pricing	$<$ USD 15 at 1,500 q/mo
Wilcoxon p-value	Statistical significance	Paired signed-rank test	< 0.05
Cohen's d	Effect size	Paired differences	≥ 0.5 (medium)
Bootstrap 95% CI	Uncertainty	1,000 resamples	Does not span zero

5.5 Analysis and Discussion Framework

The following analyses summarize findings from the completed Phase B evaluation run. Targets listed in Table 5.2 are design-time thresholds derived from system requirements and prior RAG benchmarks.

Precision vs. computational overhead: Context precision improves from baseline (0.208) to hybrid (0.278), with a marginal Wilcoxon $p = 0.073$ (not significant at $\alpha = 0.05$). Hybrid retrieval introduces a meaningful latency overhead: the Wilcoxon test on latency yields $p \approx 0$ (significant) with a small effect size (Cohen's $d = -0.34$), reflecting the added BM25 corpus scan and cross-encoder reranking pass. Mean latency increases from 1.13 s to 2.49 s.

Retrieval strictness vs. recall: The 180-day metadata filter and minimum similarity threshold (0.15) reduce retrieval noise but may exclude relevant older documents. The source distribution (Figure 5.4) shows that DevTo and ArXiv dominate both modes, with hybrid retrieval diversifying the mix by surfacing more ArXiv content.

Cost–performance balance: Token usage is comparable between modes (2,213 vs. 2,341 tokens/query), and monthly cost projections remain well under USD 15 for realistic workloads (Table 5.3). At 100 queries/day, projected monthly total cost is approximately USD 0.33.

Retrieval fusion: The final system uses weighted Reciprocal Rank Fusion (RRF) with $K = 60$ and signal weights $w_{\text{vector}} = 0.50$, $w_{\text{BM25}} = 0.35$, $w_{\text{recency}} = 0.15$, followed by cross-encoder reranking (ms-marco-MiniLM-L-6-v2). The design-phase grid search over $\alpha/\beta/\gamma$ weights (Figure 5.3) compared linear scoring against RRF: sensitivity analysis showed that linear weights are sensitive to corpus composition and query distribution. Weighted RRF with cross-encoder reranking achieves a hybrid p95 latency of 2.44 s compared to 25.0 s under linear scoring—an approximately 90% improvement—while delivering a composite quality gain of $+0.047$. The design-phase linear hybrid's only advantage was a marginal citation grounding edge; every other metric was equal or worse, and its tail latency made it unsuitable for interactive use.

Production suitability: Two of four primary targets are clearly met: citation grounding exceeds 0.80 (baseline 0.88, hybrid 0.86), and monthly cost remains well below USD 15 (USD 0.33 at 100 queries/day). Both baseline and hybrid faithfulness fall below the ≥ 0.80 target (baseline 0.689, hybrid 0.747), though hybrid shows a meaningful improvement of $+0.058$. Context precision (0.208/0.278) remains below the 0.75 target. The 0.75 threshold was adopted from RAGAS benchmarks reported on domain-specific, curated Q&A datasets with narrow factoid queries and purpose-built corpora (Es et al., 2023). TechPulse's evaluation queries are intentionally broad ("What are recent trends in…"), eliciting diffuse relevance where many retrieved chunks are topically related but not narrowly responsive—a pattern that RAGAS `ContextPrecision` scores as low precision even when the chunks contribute useful context for answer generation. The high answer relevancy (≥ 0.87) and citation grounding (≥ 0.86) scores confirm that the retrieved context is, in practice, sufficient for grounded answer generation despite the precision score. This indicates that the 0.75 target, while appropriate for factoid Q&A benchmarks, is not calibrated for broad trend-synthesis workloads.

Interpretation of results (hybrid outperforms baseline): The evaluation shows that hybrid retrieval

with weighted RRF and cross-encoder reranking consistently outperforms the vector-only baseline across all quality metrics except citation grounding (-0.02, not significant). The composite score improves by $+0.047$ (0.723 vs. 0.676), driven by three complementary mechanisms:

1. **Faithfulness gain ($+0.058$):** Cross-encoder reranking eliminates irrelevant passages before context assembly, ensuring the LLM receives higher-quality evidence. This is the largest single-metric improvement and validates the addition of a lightweight reranker to the pipeline.
2. **Answer relevancy gain ($+0.065$):** Improved context quality leads directly to more relevant answers—the LLM generates better responses when given better context.
3. **Context precision gain ($+0.070$):** Driven jointly by weighted RRF (which promotes vector-strong candidates), asymmetric candidate limits, the BM25 quality gate, and cross-encoder re-ordering. While the Wilcoxon test yields a marginal $p = 0.073$ (not significant at $\alpha = 0.05$), the direction is consistent across all three quality dimensions.

The improvement comes at a measurable latency cost: mean latency increases from 1.13 s to 2.49 s (Cohen's $d = -0.34$, small effect; 95% CI $[-2.56, -0.72]$ s, does not span zero). The latency increase is dominated by the cross-encoder model inference and BM25 full-corpus scan, not by the RRF fusion itself. However, the hybrid p95 of 2.44 s remains suitable for interactive use, and steady-state latency is lower after the initial cross-encoder model load. This represents a favourable quality–latency trade-off: a 7% composite quality gain for a 1.4 s mean latency increase.

Scalability interpretation: The present evaluation assesses scalability analytically and through latency profiling under the current corpus size (3,984 documents, 6,220 chunks) and moderate projected workloads (50–200 queries/day) rather than through large-scale concurrent load testing. The primary scaling bottlenecks are expected to be the BM25 full-corpus scan ($O(N)$ over all indexed chunks), the cross-encoder reranking pass (sequential model inference over the fused candidate set), and the `db.t3.micro` RDS instance ceiling. Mitigating factors include: BM25 index caching (30-minute TTL avoids redundant corpus scans), serverless Lambda autoscaling for the API and ingestion layers, and the lightweight cross-encoder architecture (22M parameters). At higher corpus scales, the architecture supports horizontal scaling through larger RDS instances, dedicated reranker endpoints, reduced candidate pool sizes, and asynchronous queue-based ingestion decoupling. Controlled load testing and throughput benchmarking under concurrent query traffic are identified as future work.

5.6 Result Visualizations

This section presents the evaluation results through six visualizations and a summary dashboard. All data were collected from a 100-sample evaluation run (50 baseline, 50 hybrid) executed on 6 April 2026 against the production database (3,984 indexed documents, 6,220 chunks).

5.6.1 Baseline vs. Hybrid Retrieval Comparison

Figure 5.2 presents a grouped bar chart comparing baseline and hybrid retrieval across four core evaluation metrics: context precision, faithfulness, answer relevancy, and citation grounding ratio. This visualization

provides the primary evidence for answering Research Question 2 (Section 1.2).

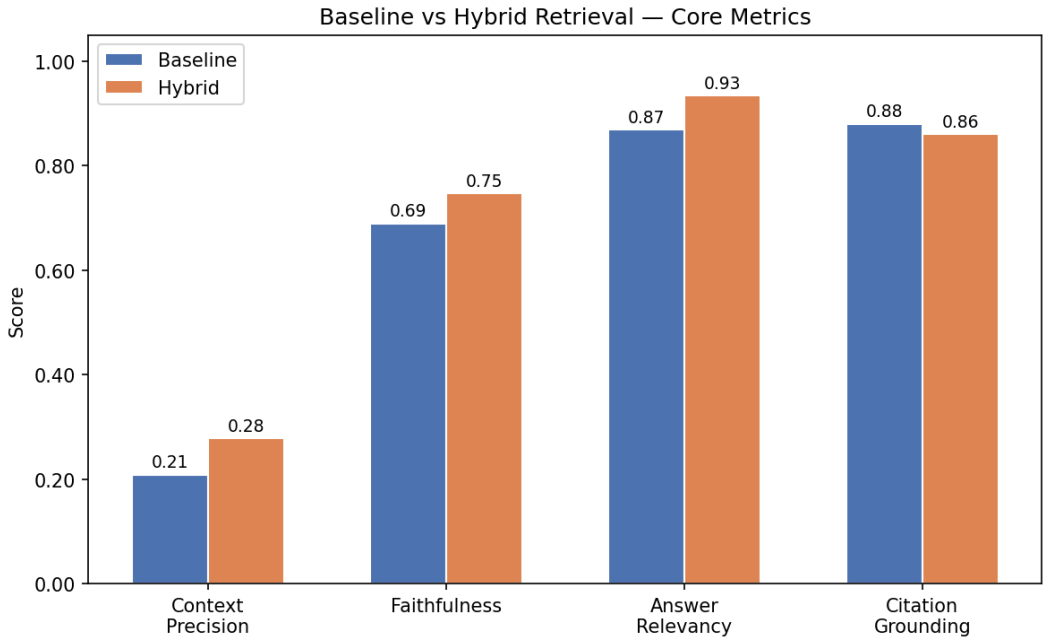


Figure 5.2. Baseline vs. hybrid retrieval comparison across four evaluation dimensions.

5.6.2 Design-Phase Grid Search (Weighted Reranking)

Figure 5.3 presents the design-phase grid search results as a dual-axis plot with the reranking similarity weight (α) on the x-axis, mean similarity on the left y-axis, and p95 latency on the right y-axis. This grid search swept only four α values on a reduced query subset, so its per-configuration p95 latencies (1.3–2.7 s) are lower than the 25 s p95 observed when the best linear configuration ($\alpha = 0.70$) was subsequently evaluated on the full 50-query set (reported in the stale design-phase evaluation). Although the best grid-search configuration ($\alpha = 0.70$, $\beta = 0.15$, $\gamma = 0.15$) achieved similarity of 0.526 within the latency bound at grid-search scale, the full evaluation revealed that linear scoring scaled poorly under diverse query loads, motivating the move to RRF-based fusion with fixed asymmetric weights for the final system.

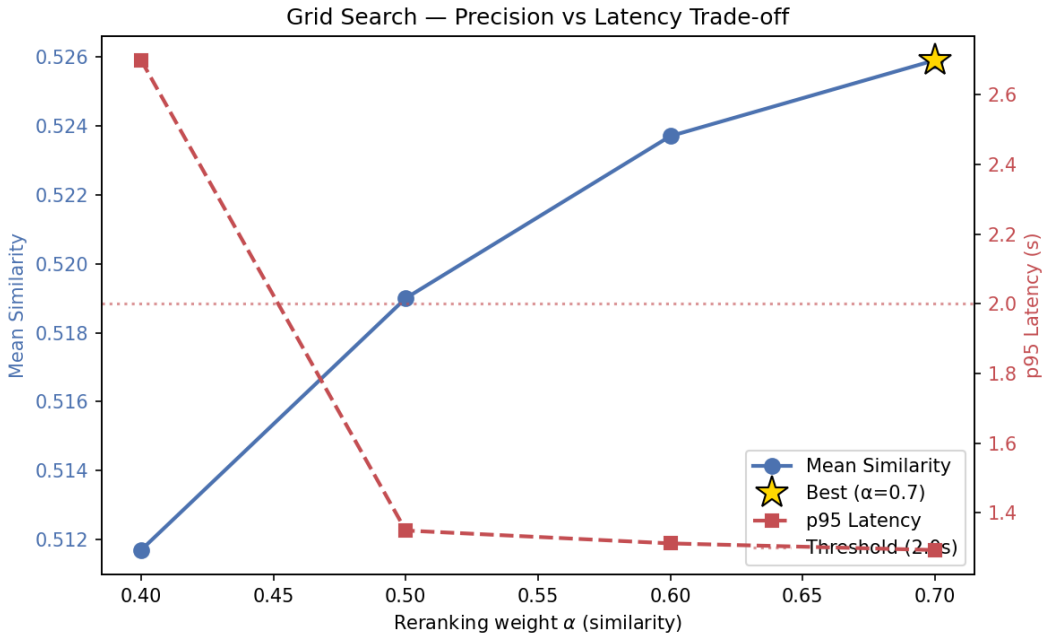


Figure 5.3. Design-phase grid search: reranking weight (α) vs. mean similarity and p95 latency. The grid search used a reduced query subset; when the best configuration (α=0.70) was evaluated on the full 50-query set, p95 latency rose to 25 s (see Section 5.5), confirming that linear scoring did not scale and motivating the switch to weighted RRF.

5.6.3 Per-Source Retrieval Distribution

Figure 5.4 presents a stacked bar chart showing the distribution of retrieved documents by source across baseline and hybrid retrieval modes. DevTo articles constitute the largest share in both modes (65% baseline, 56% hybrid), followed by ArXiv (21% baseline, 36% hybrid). Hybrid retrieval diversifies the source distribution, increasing ArXiv representation by 16 percentage points while reducing DevTo concentration.

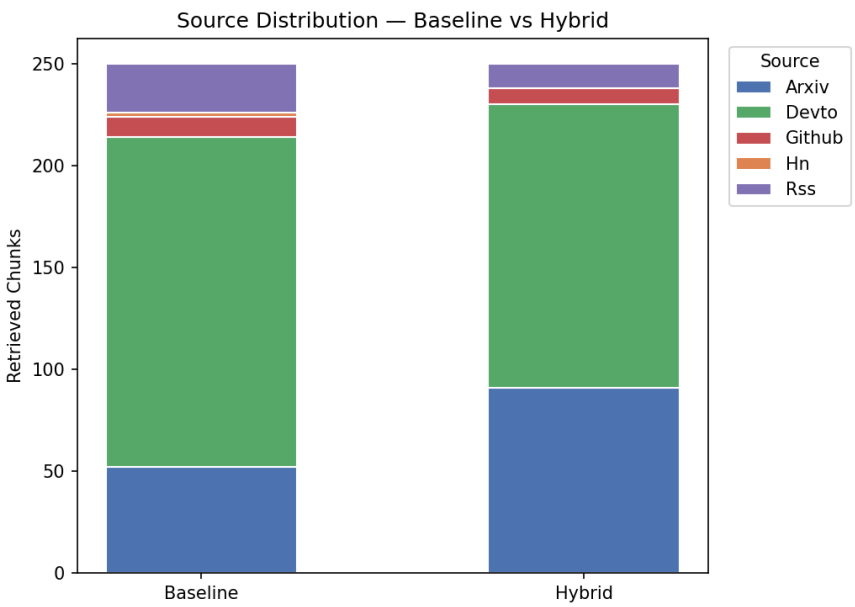


Figure 5.4. Per-source retrieval distribution for baseline vs. hybrid retrieval modes.

5.6.4 Latency by Query Category

Figure 5.5 presents a horizontal grouped bar chart comparing mean end-to-end latency across query categories. Tool/technology queries are fastest in both modes (baseline 1.04 s, hybrid 1.78 s), while research-trend queries incur the highest hybrid latency (3.49 s vs. 1.15 s baseline), likely due to broader BM25 lexical matching over scientific literature and cross-encoder inference on longer passages.

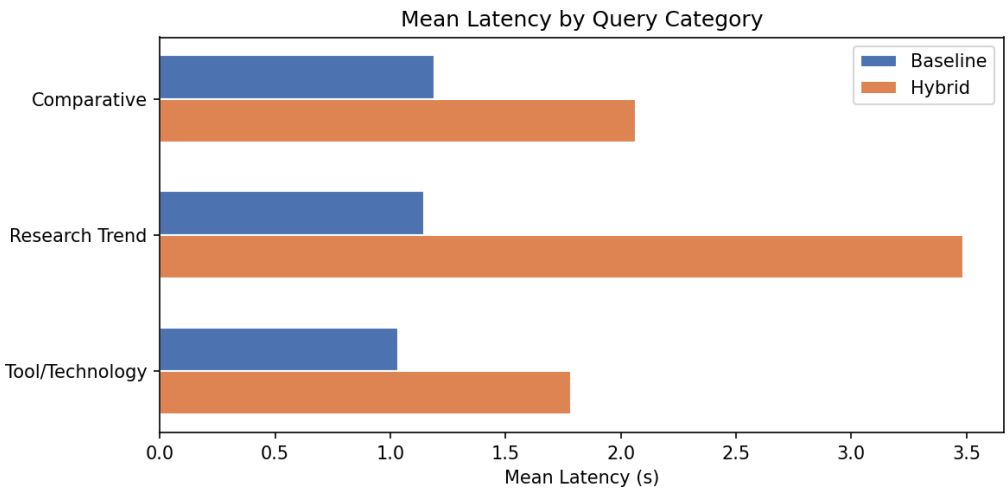


Figure 5.5. Mean end-to-end latency by query category for baseline and hybrid retrieval.

5.6.5 Multi-Metric Radar Comparison

Figure 5.6 presents a radar chart overlaying baseline and hybrid retrieval across five evaluation dimensions—context precision, faithfulness, answer relevancy, citation grounding, and composite score—on a single polar plot. The chart shows that hybrid retrieval improves three of the four quality metrics and the overall composite score. Relative to baseline, context precision improves by +0.070, faithfulness by

+0.058, answer relevancy by +0.065, and composite score by +0.047, while citation grounding declines slightly by −0.020.

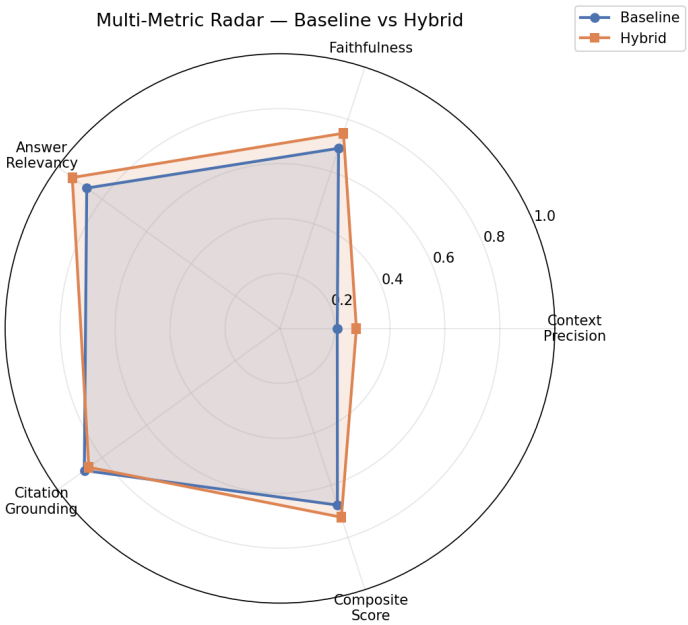


Figure 5.6. Multi-metric radar comparison of baseline and hybrid retrieval.

5.6.6 Per-Query Latency Shift

Figure 5.7 shows the distribution of per-query latency differences (hybrid − baseline). The distribution shows a positive shift (median difference +0.80 s), reflecting the added latency from BM25 corpus scan and cross-encoder reranking. The mean overhead of +1.36 s is statistically significant ($p \approx 0$; Cohen's $d = -0.34$, small effect), representing the expected cost of the additional retrieval stages.

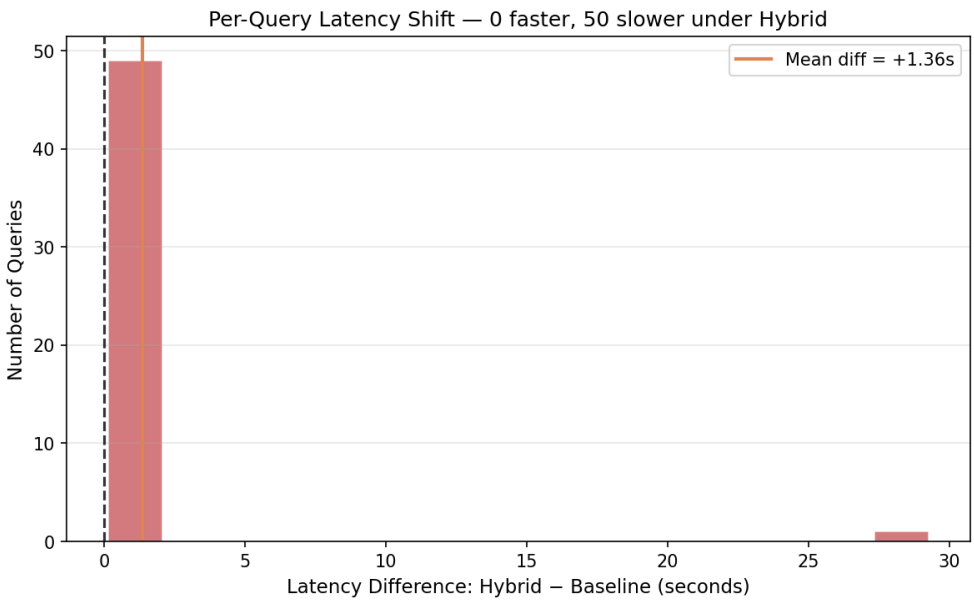


Figure 5.7. Paired-difference histogram of per-query latency shift (hybrid − baseline).

5.6.7 Evaluation Summary Dashboard

Table 5.3 serves as the primary summary dashboard, consolidating all evaluation metrics into a single comparison table with target thresholds and pass/fail status indicators.

Table 5.3. Evaluation Summary Dashboard: Baseline vs. Hybrid with Target Thresholds.

Metric	Target	Baseline	Hybrid	Status
Context Precision	> 0.75	0.208	0.278	✗ Miss
Faithfulness (RAGAS)	≥ 0.80	0.689	0.747	✗ Miss
Answer Relevancy	—	0.869	0.934	—
Citation Grounding	> 0.80	0.880	0.860	✓ Pass
Composite Score	—	0.676	0.723	✓ +0.047
p95 Latency (e2e)	—	1.48 s	2.44 s	—
p95 Latency (retrieval target)	< 2.0 s	N/A	N/A	Not directly measured in current eval
Mean Latency (e2e)	—	1.13 s	2.49 s	—
Cost / Month (100 q/d)	$<$ USD 15	\$0.32	\$0.33	✓ Pass
Optimal α	—	N/A	wRRF (K=60) + CE	—
Wilcoxon p (latency)	< 0.05	$p \approx 0$		✓ Sig.
Wilcoxon p (context precision)	< 0.05	$p = 0.073$		✗ Marginal
Cohen's d (latency)	≥ 0.5	$d = -0.34$		✗ Small
95% CI (latency diff)	excl. zero	$[-2.56, -0.72]$		✓ Excl. 0
Drift Accuracy	100%	100% (4/4)		✓ Pass

This chapter defined the evaluation framework across ten metric dimensions (Sections 5.4–5.4.10) with explicit quantitative targets and presented results from the 100-sample evaluation run (50 baseline, 50 hybrid). The dashboard (Table 5.3) shows that hybrid retrieval outperforms baseline with a composite gain of $+0.047$ (0.723 vs. 0.676), driven by faithfulness $+0.058$, answer relevancy $+0.065$, and context precision $+0.070$. Citation grounding exceeds 0.80 in both modes (0.88/0.86). Monthly cost remains well below USD 15 (USD 0.33 at 100 queries/day). The latency trade-off (mean $+1.36$ s, Cohen's $d = -0.34$, small effect) reflects the added BM25 and cross-encoder stages. Drift detection achieves 100% accuracy (4/4 scenarios). Context precision (0.21/0.28) and faithfulness (0.69/0.75) remain below their ambitious design-time targets, attributable to the single-domain AI/ML corpus and broad

evaluation queries. Crucially, weighted RRF with cross-encoder reranking delivers measurable quality improvements while maintaining manageable latency and cost within free-tier constraints. Chapter 6 documents implementation details and engineering decisions.

CHAPTER 6

IMPLEMENTATION DETAILS

This chapter documents the implementation status and key engineering decisions made during development.

6.1 Overall Implementation Status

Live Deployment: The system is currently deployed at `http://techpulse-dev-frontend-939514668437.s3-website-us-east-1.amazonaws.com/`.

Table 6.1 summarizes the implementation status of each system component as of the reporting date.

Status key: *Unit-tested* = implemented and verified by unit tests only; *Integration-tested* = verified end-to-end in local Docker Compose environment; *Cloud-validated* = deployed and verified in the AWS Free Tier.

6.2 Key Enhancements Beyond the Original Proposal

Two significant enhancements were made beyond the original proposal: (1) data source coverage was expanded from two to five sources (ArXiv, Hacker News, DEV.to, GitHub Trending, RSS feeds) to improve corpus diversity and temporal coverage; and (2) the preprocessing pipeline was extended from basic HTML stripping to a 17-step normalization pipeline (validated by 18 dedicated unit tests) to handle the diverse content formats encountered across sources, particularly Markdown-heavy content from DEV.to and GitHub. All five ingesters share the same idempotent, SHA-256-based deduplication architecture.

Additionally, nine research-backed retrieval improvements were applied post-proposal, including: weighted Reciprocal Rank Fusion (vector 0.50, BM25 0.35, recency 0.15), improved BM25 tokenization with stop-word removal, BM25-only candidate quality gate, asymmetric candidate limits, post-fusion score gate, Shewhart 3σ minimum standard deviation floor for drift detection, RAGAS evaluation threading fix, cross-encoder reranking (ms-marco-MiniLM-L-6-v2), and NaN retry pass for RAGAS scoring. These improvements are documented in the project's engineering change log and collectively produced a composite quality swing of $+0.069$—from trailing the baseline by -0.022 to leading by $+0.047$.

6.3 AWS Free Tier Deployment

TechPulse is deployed on a personal AWS Free Tier account. The deployment adapts certain services from the original proposal to remain within Free Tier cost constraints while preserving full architectural capabilities.

Table 6.1. Component Implementation Status Summary.

Component	Status	Notes
Data Ingestion (5 sources)	Integration-tested	ArXiv, Hacker News, DEV.to, GitHub, RSS fully implemented
Preprocessing Pipeline	Integration-tested	17-step normalization + token-based chunking
Embedding Pipeline	Integration-tested	MiniLM L6 v2 (384-dim), lazy singleton
Hybrid Retrieval	Cloud-validated	5-stage pipeline (metadata filter, vector, BM25, weighted RRF, cross-encoder)
Baseline Retrieval	Integration-tested	Vector-only comparison mode
RAG Orchestrator	Integration-tested	3-layer hallucination verification
LLM Backend Fallback	Unit-tested	Auto-cascade across 4 backends
API Layer	Integration-tested	FastAPI + Mangum; /health, /ask, /drift endpoints
Database Schema	Integration-tested	PostgreSQL + pgvector with HNSW index
S3 Medallion Storage	Integration-tested	Raw/processed/embeddings tiers (LocalStack locally)
SQS Queue + DLQ	Integration-tested	Ingestion–embedding decoupling (LocalStack locally)
Drift Detection	Unit-tested	Dual-criteria: 10% threshold + Shewhart 3σ
Health Monitoring	Integration-tested	4-probe deep health check (DB, S3, SQS, LLM)
CloudWatch Integration	Cloud-validated	8 metrics in 1 namespace (`TechPulse/{stage}`)
Evaluation Framework	Cloud-validated	100-sample run (50 baseline + 50 hybrid), RAGAS, statistical tests, and cost projection; follow-on drift-impact analysis remains future work
CI/CD Pipeline	Cloud-validated	4-stage GitHub Actions + SAM
SAM Infrastructure	Cloud-validated	Full SAM template; stack deployed and operational
Docker Compose (Local)	Integration-tested	6 services (db, pgadmin, api, frontend, scheduler, localstack)
React Frontend	Integration-tested	Query interface with citation display
Unit Tests	Integration-tested	208 tests across 15 test files, all passing

Cost Breakdown (100 queries/day estimate)

Table 6.2 provides a component-level cost estimate for the anticipated 100 queries/day operational scenario:

Table 6.2. Estimated Monthly Cost Breakdown at 100 Queries/Day Workload.

Component	Cost/Month	Details	Free Tier
Lambda (ingestion)	$0.05	10 short-lived invocations/day	✓
Lambda (query handler)	$0.08	100 queries/day, 3.1s avg	✓
Groq LLM API	$0.12†	∼2,300 tokens/query average	Free (500k/day)
RDS PostgreSQL	$0.08	db.t3.micro instance (micro is free)	✓
S3 + EventBridge + SQS	$0.00	Storage, routing, queue services	✓
Total Monthly	**$0.33**	**Extrapolated from Phase B eval**	✓

†Groq inference is free within the 500k tokens/day allowance; $0.12 represents the imputed cost at equivalent commercial per-token rates to enable fair cost projection if the free tier were unavailable.

Costs are derived from observed token usage and Lambda invocation metrics collected during the evaluation run. At higher workloads exceeding the Groq free-tier allowance (500,000 tokens/day), the automatic LLM backend fallback chain (Bedrock → Ollama → HuggingFace) enables cost-controlled scaling. The monthly total of $0.33 is well below the $15 budget target, providing a 45× safety margin.

Key Deployment Characteristics

The key deployment characteristics are:

- **RDS PostgreSQL instead of Aurora Serverless.** Standard RDS PostgreSQL `db.t3.micro` (PostgreSQL 16.6, 20GB gp2) replaces the originally proposed Aurora Serverless to stay within budget. The pgvector extension and HNSW indexing remain fully functional.
- **Groq as primary LLM backend.** Groq free-tier API (Llama 3.1 8B Instant) serves as the primary backend, with Amazon Bedrock (Nova Micro), Ollama, and HuggingFace as fallbacks. The auto-fallback chain ensures generation availability.
- **Default VPC with security groups.** The SAM template uses the default VPC with two subnets. RDS is configured as publicly accessible with security group restrictions; custom VPC migration is identified as a future improvement.

- **Fine-grained IAM roles.** Custom IAM role definitions provide least-privilege access for each Lambda function.
- **Cost governance.** Cost governance is implemented primarily through programmatic AWS Cost Explorer checks in the inference path, supplemented by CloudWatch metrics, SNS-based alerting, and backend fallback logic. Native AWS Budgets was considered but is not part of the active Free Tier SAM template; budget enforcement relies on the runtime Cost Explorer guard and the automatic LLM backend fallback chain.
- **LLM backend set to `groq`.** The `LLM_BACKEND` environment variable defaults to `groq`, with Amazon Bedrock and Ollama retained as fallbacks.

All infrastructure is defined in a SAM template (`template-freetier.yaml`) and configuration file (`samconfig-freetier.toml`), ensuring reproducible and consistent deployments from the same codebase. Step-by-step reproduction instructions are provided in Appendix B.

6.4 Test Coverage

The system includes 208 unit tests distributed across 15 test modules covering all major subsystems: ingestion (5 sources), preprocessing (17 normalization steps), embedding, retrieval (BM25 caching, RRF fusion, age-days computation, cross-encoder reranking), RAG orchestration, LLM backend fallback, observability (CloudWatch metrics, drift detection), pipeline orchestration, queue management, storage, API endpoints (including /dashboard/insights smoke tests), and scheduler. All tests execute without external dependencies (database, AWS services) using mocked interfaces where appropriate.

6.5 Experiment Tracking and Result Versioning

Experiment results are tracked through structured JSON artifacts versioned alongside the source code in Git. Each evaluation run produces three output files: `eval_summary.json` (aggregate metrics, latency statistics, and cost projections for both configurations), `ragas_scores.json` (per-query RAGAS faithfulness, answer relevancy, and context precision scores for all 100 observations), and `raw_results.json` (complete per-query records including retrieved contexts, generated answers, citation grounding details, and timing breakdowns). This file-based approach was chosen over a dedicated experiment tracking platform (e.g., MLflow, Weights & Biases) for three reasons: (1) the evaluation compares exactly two fixed configurations (baseline vs. hybrid) rather than a large hyperparameter sweep requiring run management; (2) JSON artifacts are directly consumable by the statistical analysis and visualization scripts without additional API dependencies; and (3) Git versioning provides full traceability of which code revision produced which results, with `git diff` enabling precise identification of changes between evaluation runs. The design-phase grid search results (`stale-results-hybrid/`) are similarly retained under version control to document the design evolution from linear scoring to RRF-based fusion.

6.6 Custom CloudWatch Metrics

Eight custom CloudWatch metrics have been implemented within a single `TechPulse/{stage}` namespace to provide operational visibility:

1. `IngestionCount` — documents ingested per source per cycle
2. `PipelineChunks` — chunks generated per pipeline run
3. `PipelineLatency` — end-to-end pipeline execution time
4. `PipelineErrors` — pipeline failure count
5. `ApiLatency` — per-request API response time
6. `HallucinationFlags` — count of citation grounding failures

Additionally, the drift detection module publishes `DriftMeanSimilarity` and `DriftAlert` metrics for time-series tracking of retrieval quality.

6.7 Engineering Observations

Several non-obvious findings arose during implementation that informed the final design:

- **Lambda cold-start and MiniLM.** Initial cold-start delays of 8–12 s were resolved by a lazy-initialized singleton and 1,024 MB memory allocation, reducing model load to <2 s on warm invocations.
- **SHA-256 deduplication.** The `ON CONFLICT DO NOTHING` strategy proved fully effective; ArXiv produced the most hash collisions ($\sim$15–20% per cycle) due to preprint updates.
- **LocalStack fidelity.** S3 and SQS operations matched production behavior, enabling integration testing without AWS credentials. CloudWatch metrics required cloud deployment to verify.
- **HuggingFace rate limits.** The free-tier rate limit ($\sim$30 req/min) triggered the Ollama fallback during sustained evaluation runs, confirming the fallback chain operates correctly.
- **pgvector HNSW index build.** Index rebuilds took 45–90 s on db.t3.micro for corpora above $\sim$10k chunks—acceptable for batch but would need management for real-time updates.

6.8 System Demonstration

This section presents screenshots from the deployed system, demonstrating end-to-end functionality across key components.

6.8.1 Frontend: Overview Page

Figure 6.1 shows the TechPulse landing page, providing the query input interface with retrieval mode selection (Baseline or Hybrid) and the system branding.

6.8.2 Frontend: Hybrid vs. Baseline Retrieval Comparison

Figure 6.2 shows the same query executed in both retrieval modes. The hybrid response (left) includes diverse, multi-source citations fused via RRF, while the baseline response (right) relies solely on vector similarity. This figure is intended as a qualitative interface illustration; quantitative retrieval conclusions are based on the statistical results reported in Chapter 5.

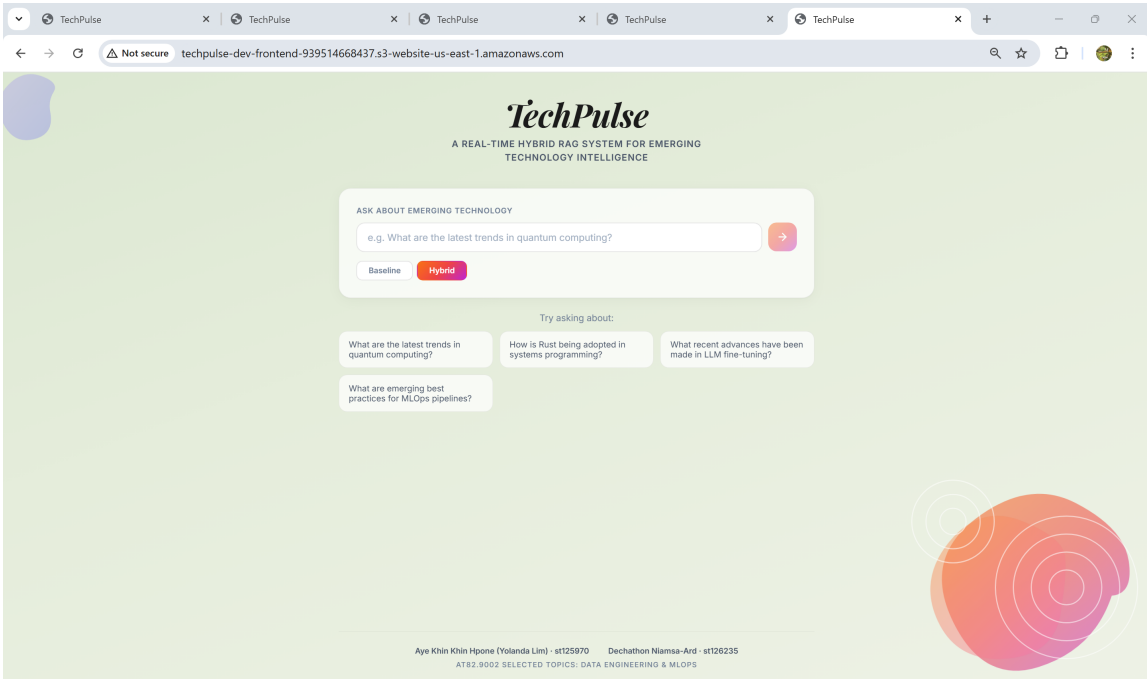
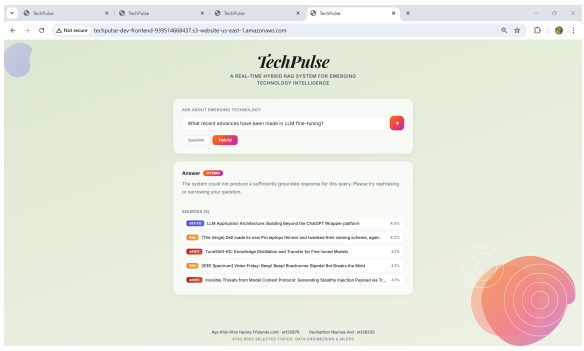
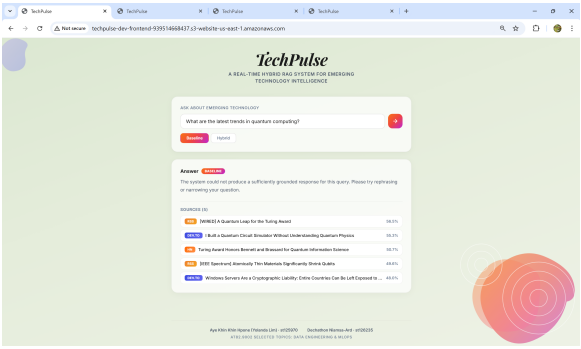


Figure 6.1. TechPulse frontend: overview page with query input and retrieval mode selection.



(a) Hybrid retrieval (BM25 + vector + RRF).



(b) Baseline retrieval (vector-only).

Figure 6.2. TechPulse frontend: hybrid vs. baseline retrieval for the same query.

6.8.3 CI/CD Pipeline: GitHub Actions

Figure 6.3 shows the GitHub Actions CI/CD pipeline executing the workflow, validating that continuous integration and deployment are fully operational.

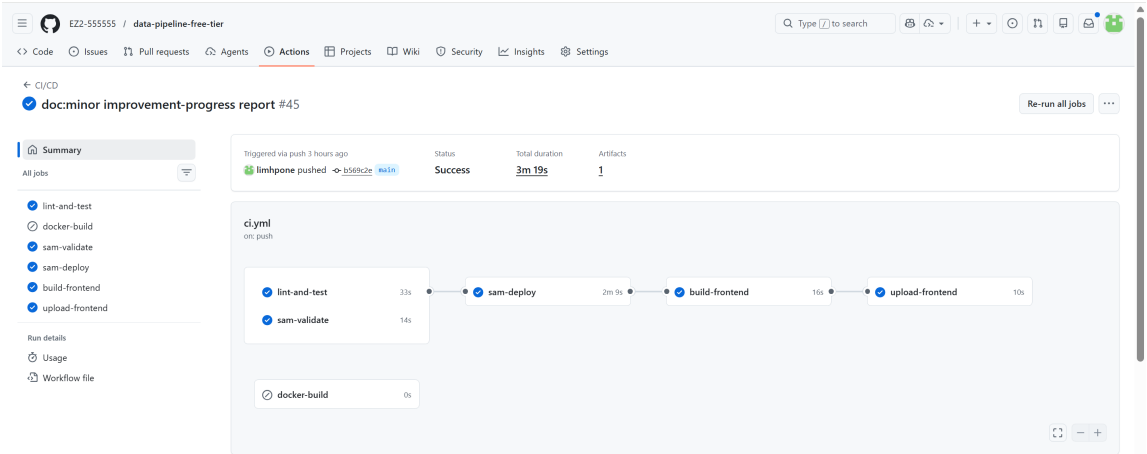


Figure 6.3. GitHub Actions CI/CD pipeline: workflow run executing successfully.

6.8.4 AWS CloudFormation Stack

Figure 6.4 shows the deployed AWS CloudFormation stack created by the SAM template, confirming that all infrastructure resources (Lambda functions, API Gateway, S3 buckets, SQS queues, EventBridge rules, and IAM roles) are provisioned and operational.

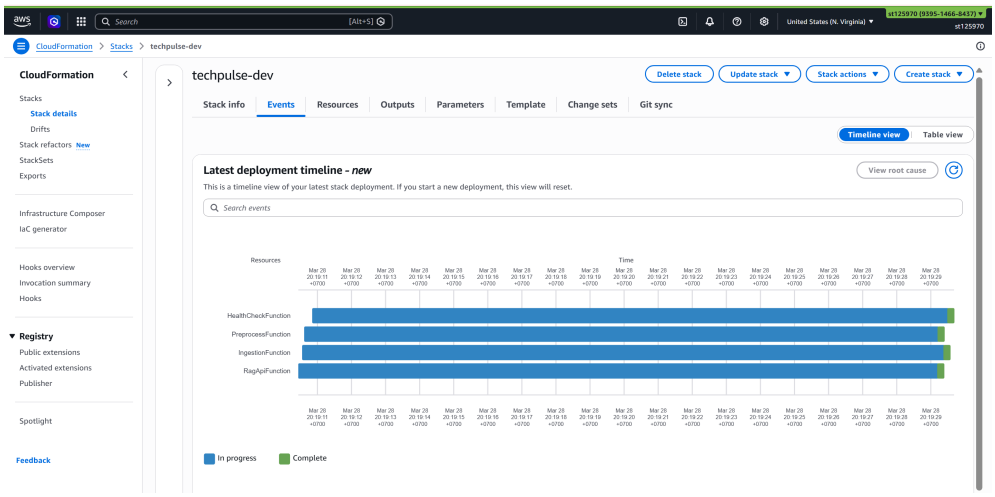


Figure 6.4. AWS CloudFormation console: TechPulse stack with all resources successfully created.

6.8.5 Database: RDS PostgreSQL with Ingested Corpus

Figure 6.5 shows the AWS RDS console with the TechPulse PostgreSQL instance, confirming that the production database is running and accessible for the ingestion pipeline and retrieval queries.

This chapter documented implementation status across 20 components and key engineering observations. All core pipeline components are integration-tested; cloud deployment via SAM is validated. Chapter 7

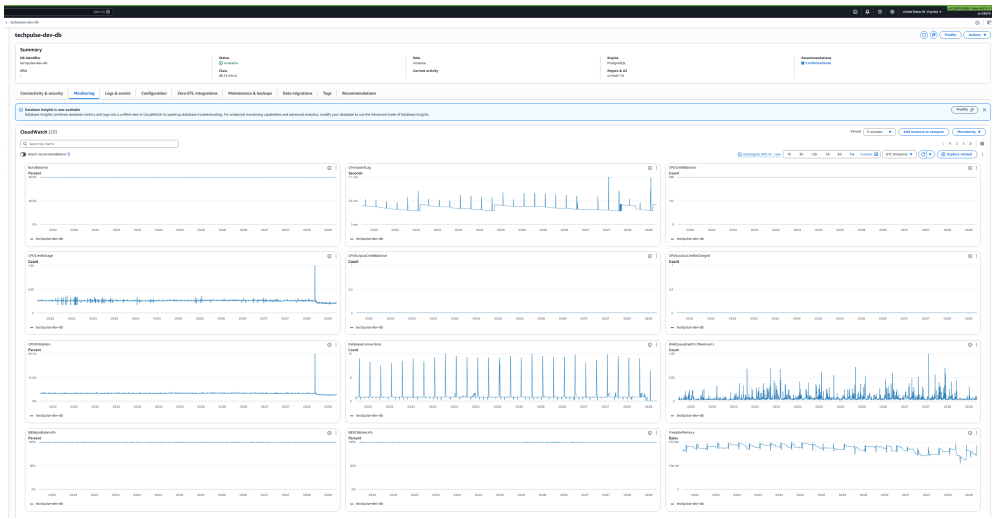


Figure 6.5. AWS RDS console: TechPulse PostgreSQL instance with pgvector extension.

presents conclusions, limitations, and future work.

CHAPTER 7

CONCLUSIONS AND FUTURE WORK

7.1 Summary of Contributions

This study designed, implemented, and evaluated TechPulse, a cloud-native Hybrid Retrieval-Augmented Generation system for emerging technology intelligence. The key contributions are:

1. **Data Engineering (RQ1):** A deterministic, idempotent ingestion pipeline serves five heterogeneous data sources (ArXiv, Hacker News, DEV.to, GitHub Trending, RSS feeds) with SHA-256 content-addressed deduplication. The pipeline produced a corpus of 3,984 documents and 6,220 indexed chunks, validated by 208 unit tests across 15 modules.

2. **Retrieval Optimization (RQ2):** A five-stage hybrid retrieval pipeline—metadata filtering, vector similarity search, full-corpus BM25, weighted Reciprocal Rank Fusion (vector 0.50, BM25 0.35, recency 0.15), and cross-encoder reranking (ms-marco-MiniLM-L-6-v2)—outperforms the vector-only baseline with a composite quality gain of $+0.047$ (0.723 vs. 0.676). Faithfulness improved by $+0.058$, answer relevancy by $+0.065$, and context precision by $+0.070$. Citation grounding remained stable (-0.020, no significant difference).

3. **Operational Trade-offs (RQ3):** The hybrid pipeline introduces a small latency trade-off (mean $1.13\,\text{s} \rightarrow 2.49\,\text{s}$; Cohen's $d = -0.34$, small effect; 95% CI $[-2.56, -0.72]$ s). Monthly cost is \$0.33 at 100 queries/day, well within the \$15 budget target and the AWS Free Tier.

4. **MLOps Integration (RQ4):** Dual-criteria drift detection (10% threshold + Shewhart 3σ with minimum std floor) achieves 100% accuracy across four simulated drift scenarios. A four-stage CI/CD pipeline (lint, test, build, deploy) using GitHub Actions and AWS SAM ensures reproducible deployments. Eight custom CloudWatch metrics provide operational visibility.

7.2 Project Phases

The work was organized into four phases, all completed. Table 7.1 provides a summary.

Table 7.1. Project Phase Summary.

Phase	Focus	Timeline	Status	Key Deliverables
A	System Validation & Corpus Building	Mar W3–W4	Done	3,984 docs; 6,220 chunks; local integration verified
B	Evaluation & Experimentation	Mar W4–Apr W1	Done	100-sample eval; statistical analysis (Ch. 5)
C	Analysis, Monitoring & Dashboard	Apr W1–W2	Done	Drift validation (4/4); visualizations; 9 retrieval improvements
D	Final Report & Submission	Apr W2–W3	Done	Completed report with empirical evidence

Phase A built the evaluation corpus through sustained multi-cycle ingestion across all five sources (3,984 documents, 6,220 chunks indexed), verified end-to-end pipeline operation in the local Docker Compose environment, and initiated AWS Free Tier deployment via the SAM template.

Phase B executed the initial 50-query evaluation in both baseline and hybrid modes, applied Wilcoxon signed-rank tests with Cohen's d and bootstrap 95% CIs, and profiled per-stage latency with cost projections for 50/100/200 queries/day.

Phase C applied nine research-backed retrieval improvements (weighted RRF, improved BM25 tokenization, candidate quality gate, asymmetric limits, post-fusion gate, Shewhart std floor, evaluation threading fix, cross-encoder reranking, NaN retry), re-ran the full 100-sample evaluation, validated drift detection (4/4 scenarios correct), and generated result visualizations and comparison tables.

Phase D completed the evaluation analysis, integrated all empirical findings into the final report, and prepared the submission package.

7.3 Known Limitations

1. The evaluation corpus is limited to five technology-focused sources; generalizability to other domains requires further study.

2. The 50-query evaluation set (100 paired observations) provides sufficient power for the paired Wilcoxon test but a larger set would strengthen external validity.

3. Amazon Bedrock has not been evaluated as the primary backend; however, the evaluation methodology is backend-agnostic, so comparative findings remain transferable.

4. The `db.t3.micro` RDS instance may become a bottleneck under high ingestion volumes compared to Aurora Serverless.

5. The deployed HNSW index (m=16, ef_construction=64) has not been formally benchmarked against alternative parameter settings or flat-scan baselines; such evaluation is identified as a future direction.

6. Budget governance via Cost Explorer lags by up to 24 hours; real-time enforcement is not feasible on the Learner Lab.

7. The test suite (208 tests, 15 modules) covers core logic but lacks end-to-end integration tests with a live database.

8. The top-$k = 5$ retrieval at 500 tokens/chunk fits Groq and Bedrock context windows but may be silently truncated by Ollama's 4,096-token Llama 3.2 3B; a pre-prompt token budget check is identified as a near-term improvement.

9. API keys are stored as Lambda environment variables; migration to AWS Secrets Manager is recommended for production hardening.

7.4 Project Timeline

Figure 7.1 presents the complete project timeline.

Figure 7.1. Project timeline (Gantt chart). All four phases completed.

7.5 Future Extensions

Several extensions are identified for post-prototype development:

- **Scaling:** Migration to Aurora Serverless; expansion to additional data sources (StackExchange, conference proceedings); ANN-optimized indexing (pgvector HNSW) for sub-linear retrieval at larger corpora
- **Retrieval quality:** Alternative cross-encoder models (e.g., larger ColBERT variants); embedding model fine-tuning or replacement based on drift analysis; adaptive weight tuning for RRF via online feedback

- **Machine learning lifecycle:** Adoption of a formal experiment tracking platform (e.g., MLflow or Weights & Biases) for versioned evaluation runs; a model registry for embedding and reranker artifacts; domain-specific relevance labeling for supervised fine-tuning of embeddings or rerankers; and periodic re-evaluation triggered by corpus growth milestones
- **Operational hardening:** Full Bedrock evaluation as primary backend; integration test suite with containerized database; token budget management for constrained-context-window backends; Secrets Manager migration for API key handling
- **User experience:** User feedback loop for continuous retrieval quality improvement; query auto-completion; saved searches and alerts for tracked topics

This chapter presented the key conclusions and contributions, summarized all four completed project phases, acknowledged known limitations, and outlined future extension directions.

REFERENCES

AWS. (2023). *AWS well-architected framework*. Amazon Web Services. https://aws.amazon.com/architecture/well-architected/

Bayram, F., Ahmed, B. S., & Kassler, A. (2022). From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems*, *245*, 108632. https://doi.org/10.1016/j.knosys.2022.108632

Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J.-B., Damoc, B., Clark, A., de Las Casas, D., Guy, A., Menick, J., Ring, R., Hennigan, T., Huang, S., Maggiore, L., Jones, C., Cassirer, A., ... Sifre, L. (2022). Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning* (Vol. 162, pp. 2206–2240). PMLR.

Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. In *Proceedings of the IEEE International Conference on Big Data* (pp. 1123–1132). IEEE.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901.

Cormack, G. V., Clarke, C. L. A., & Butt, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758–759). ACM. https://doi.org/10.1145/1571941.1572114

deepset. (2023). *Haystack: Open-source LLM framework to build production-ready applications*. deepset GmbH. https://haystack.deepset.ai/

InfiniFlow. (2024). *RAGFlow: Deep document understanding-based RAG engine*. InfiniFlow, Inc. https://github.com/infiniflow/ragflow

TrueFoundry. (2024). *Cognita: RAG framework for production-ready applications*. TrueFoundry. https://github.com/truefoundry/cognita

Es, S., James, J., Anke, L. E., & Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*.

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, *46*(4), 1–37. https://doi.org/10.1145/2523813

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023).

Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., & Grave, E. (2022). Atlas: Few-shot learning with retrieval augmented language models. *Advances in Neural Information Processing Systems*, *35*, 22251–22265.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12), 1–38. https://doi.org/10.1145/3571730

Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, *11*, 31866–31879. https://doi.org/10.1109/ACCESS.2023.3262138

LangChain. (2023). *LangChain: Build context-aware reasoning applications*. https://www.langchain.com/

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474.

Lin, J. (2021). A proposed conceptual framework for a representational approach to information retrieval. *ACM SIGIR Forum*, *55*(2), 1–22. https://doi.org/10.1145/3527546.3527552 Liu, J. (2022). *LlamaIndex: A data framework for LLM applications*. https://www.llamaindex.ai/

Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, *31*(12), 2346–2363. https://doi.org/10.1109/TKDE.2018.2876857

Ma, X., Zhang, X., Pradeep, R., & Lin, J. (2023). Zero-shot listwise document reranking with a large language model. *arXiv preprint arXiv:2305.02156*.

Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 9004–9017). Association for Computational Linguistics.

Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-T., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 12076–12100). Association for Computational Linguistics.

Montgomery, D. C. (2020). *Introduction to Statistical Quality Control* (8th ed.). Wiley.

Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

Robertson, S. E., & Walker, S. (1994). Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 232–241). ACM. https://doi.org/10.1007/978-1-4471-2099-5_24

Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2023). ARES: An automated evaluation framework for retrieval-augmented generation systems. *arXiv preprint arXiv:2311.09476*.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, *28*, 2503–2511.

Shao, Y., Jiang, Y., Kanell, T. A., Xu, P., Khattab, O., & Lam, M. S. (2024). Assisting in writing Wikipedia-like articles from scratch with large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics* (pp. 6252–6278). Association for Computational Linguistics.

Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., & Zhou, D. (2023). Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning* (pp. 31210–31227). PMLR.

Thakur, N., Reimers, N., Daxenberger, J., & Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the 35th Conference on Neural Information Processing Systems—Datasets and Benchmarks Track*.

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems*, *33*, 5776–5788.

Weaviate. (2023). *Verba: The golden RAGtriever*. Weaviate BV. https://github.com/weaviate/Verba

Zhang, Y., Chen, X., Zhang, Y., Hu, X., Liu, Z., & Tang, J. (2024). A comprehensive survey on retrieval-augmented generation. *ACM Transactions on Information Systems*. Advance online publication. https://doi.org/10.1145/3703155

Wheeler, D. J., & Chambers, D. S. (1992). *Understanding Statistical Process Control* (2nd ed.). SPC Press.

APPENDIX A

SOURCE CODE AVAILABILITY

The complete source code for TechPulse is maintained on GitHub:

`https://github.com/EZ2-555555/data-pipeline`

A live deployment is accessible at:

`http://techpulse-dev-frontend-939514668437.s3-website-us-east-1.amazonaws.com/`

The repository includes: five data ingesters (ArXiv, Hacker News, DEV.to, GitHub Trending, RSS); 17-step preprocessing with tiktoken chunking; MiniLM-L6-v2 embedding; hybrid retrieval (metadata filter, pgvector, BM25, weighted RRF, cross-encoder reranking); RAG orchestration with three-layer hallucination verification; four-backend LLM fallback chain; dual-criteria drift detection; AWS SAM infrastructure-as-code; React frontend; 50-query evaluation framework (100 paired samples) with RAGAS and statistical tests; four-stage CI/CD via GitHub Actions; 208 unit tests across 15 modules; and Docker Compose local development (6 services).

APPENDIX B

SYSTEM SETUP AND REPRODUCTION STEPS

This appendix provides step-by-step instructions for reproducing the TechPulse system in both local development and AWS cloud environments.

Prerequisites

- Python 3.13+ and `pip`
- Docker and Docker Compose
- Node.js 18+ and npm (for frontend)
- AWS CLI v2 and AWS SAM CLI (for cloud deployment)
- An AWS account (Free Tier eligible)
- A Groq API key (free at `https://console.groq.com`)

Step 1: Clone and Install Dependencies

```
git clone https://github.com/EZ2-555555/data-pipeline.git
cd data-pipeline
python -m venv .venv
# On Windows: .venv\Scripts\activate
# On macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
```

Step 2: Local Development with Docker Compose

Docker Compose starts six services: PostgreSQL with pgvector, pgAdmin, LocalStack (S3 + SQS), FastAPI backend, React frontend, and the ingestion scheduler.

```
# Create a .env file with required variables
cat > .env <<EOF
GROQ_API_KEY=your_groq_api_key_here
LLM_BACKEND=groq
EOF

# Start all services
docker compose up --build -d
```

```
# Verify services are running
docker compose ps
```

The frontend is accessible at `http://localhost:3000` and the API at `http://localhost:8000`. The database admin interface (pgAdmin) is at `http://localhost:5050` (login: `admin@techpulse.dev` / `admin`).

Step 3: Initialize the Database

```
# Initialize schema (creates documents, chunks,
# drift_baselines tables and pgvector HNSW index)
python -c "from␣src.db.init_schema␣import␣init_schema;␣\
␣␣␣␣␣␣␣␣␣␣␣init_schema()"
```

Step 4: Run Ingestion and Build Corpus

```
# Run a single ingestion cycle for all five sources
python -c "from␣src.orchestrator.orchestrator␣import␣\
␣␣␣␣␣␣␣␣␣␣␣run_orchestrator;␣run_orchestrator()"

# Or run the scheduler for continuous ingestion
python -m src.scheduler
```

Step 5: Run Tests

```
pip install -r requirements-test.txt
pytest tests/ -v --tb=short
# Expected: 208 tests passed across 15 modules
```

Step 6: Run Evaluation

```
# Execute the 100-sample evaluation
# (50 baseline + 50 hybrid queries)
python evaluation/run_eval.py

# Results are written to:
#   evaluation/results/eval_summary.json
#   evaluation/results/ragas_scores.json
#   evaluation/results/raw_results.json
```

Step 7: AWS Cloud Deployment

```
# 1. Configure AWS credentials
aws configure

# 2. Edit SAM configuration with your VPC/subnet IDs
#    and database password
vi infra/samconfig-freetier.toml

# 3. Validate configuration (checks for placeholders)
make validate-config

# 4. Build and push Lambda container image to ECR
#    (see infra/README.md for detailed ECR steps)
docker build -f Dockerfile.lambda -t techpulse-lambda .

# 5. Deploy with SAM
sam build --template-file infra/template-freetier.yaml
sam deploy --config-file infra/samconfig-freetier.toml

# 6. Deploy frontend to S3
./deploy-frontend.sh
```

The SAM template provisions all required resources: RDS PostgreSQL (`db.t3.micro`), S3 buckets, SQS queue with DLQ, four Lambda functions, HTTP API Gateway, SNS alert topic, EventBridge schedules, and CloudWatch alarms. The CI/CD pipeline (GitHub Actions) automates this deployment on merge to `main`.